

PTT0 课程简介

1、一个程序的执行过程有哪些主要步骤？

程序执行是一个从“用户发起”到“硬件响应”的完整生命周期，主要包含以下步骤：

解析与定位：OS 在磁盘中找到目标程序，检查其类型是否为可执行文件，读取首部信息，确定代码和数据在文件中的位置并计算磁盘块地址。

创建进程：OS 创建一个新的进程，并将可执行文件映射到该进程结构中。

调度与上下文：OS 为该程序设置 CPU 上下文环境，调度选中该进程后，跳到程序开始处准备执行。

内存分配（缺页异常）：执行第一条指令时通常不在内存，发生缺页异常(Page Fault)。OS 分配物理内存页，将代码从磁盘读入内存，然后恢复程序的执行。

系统调用：程序执行到特权指令（如 puts 输出字符串）时，触发系统调用，将控制权交还给 OS。

设备交互：OS 找到对应的设备控制进程（如显卡驱动），进程将数据转换为硬件可理解的信号（如将字符转为像素写入帧缓冲区）。

硬件响应：硬件（如视频接口和显示器）接收信号并完成最终物理动作（如屏幕发光显示文本）。

2、操作系统内核中的主要功能模块有哪些？哪些与具体的体系结构相关？

a. 内核的主要功能模块：

系统调用：提供应用与内核交互的接口。

系统调度与管理：负责进程/线程创建、切换、调度与同步。

内存管理：负责物理及虚拟内存分配、地址映射、缺页处理。

中断处理：处理硬件中断、异常和陷阱。

文件系统：管理磁盘文件与目录结构。

I/O 管理 / 设备管理 (Device Drivers)：管理各种外部设备。

b. 与具体体系结构（如 x86、RISC-V）强相关的模块：

中断/异常处理底层：不同架构的陷阱/异常触发机制和寄存器（如 RISC-V 的 mepc 寄存器、mret 指令）完全不同。

内存管理底层：页表格式、TLB 刷新、存储器物理地址和虚拟地址转换（MMU）强依赖于 CPU 架构。

进程管理的上下文切换：保存和恢复 CPU 寄存器状态（Context Switch）由特定架构的汇编指令完成。

设备驱动底层：对设备接口寄存器和缓冲区的物理读写操作与具体主板和芯片总线强相关。

3、操作系统是怎样与硬件打交道的？【硬件接口】

1)面对物理机器界面：操作系统最底层直接与“物理机器界面”对接。

2)通过寄存器与指令交互：OS 通过设置物理地址、读写设备接口寄存器和设备接口缓冲区来向硬件发送指令。比如操作软盘控制器（如 NEC PD765），需要写入特定的 16 条命令，并装入对应长度（1~9 字节）的硬件参数（扇区、磁头、轨道地址等）。

3)获取硬件状态：OS 读取寄存器返回的特定字

节（状态及出错字段）来判断硬件操作是否成功，并利用硬件中断机制获取设备完成信号。

4)硬件抽象与屏蔽（OS 的核心价值）：因为直接与硬件打交道极其复杂繁琐、代码量大且不通用，操作系统通过编写底层的“设备驱动程序”，将这些硬件相关的操作屏蔽和统一起来。向上为应用程序提供“硬件抽象”（如把磁盘抽象为“文件”），提供有限且标准化的基本服务，从而将复杂的物理机器变成一台等价的、易于编程的扩展机器（虚拟机）。

PTT1 操作系统概述

1、文件系统与块设备之间的 Buffer Cache 的作用是什么？Buffer Cache 的工作原理是什么？

作用：

减少对物理设备的访问次数,提高 I/O 性能

协调 CPU 与 I/O 设备之间的速度差异

支持数据的读写缓冲,实现数据共享

工作原理：

当进程请求读取数据时,系统首先检查 Buffer Cache 中是否有所需数据:

a.如果有(命中),直接从 Buffer Cache 返回数据

b.如果没有(未命中),从物理设备读取数据到 Buffer Cache,再返回给进程

c.写操作时,数据先写入 Buffer Cache,再由系统决定何时写回物理设备。

2、UNIX 的名字来历(猜测)

UNIX 名字可能来源于 MULTICS 的简化和双关语:

MULTICS 是一个复杂的分时系统。

(Multiplexed Information and Computing Service)

UNIX(Uniplexed Information and Computing Service)表示简化版的 MULTICS

也有说法是“UNIX”是“eUnuchs”(阉割版的 MULTICS)的变体

3、批处理操作系统的区别

- 单道批处理:一次只能执行一个作业,作业按顺序依次执行,前一个作业完成后才能执行下一个

- 多道批处理:内存中同时存放多个作业,CPU 在这些作业之间切换执行,提高了 CPU 利用率和系统吞吐量

4、SPOOLing 技术

工作原理：

a.作业进入到磁盘上的输入井

b.按某种调度策略选择几种搭配得当的作业,并调入内存

c.作业运行的结果输出到磁盘上的输出井

d.结果从磁盘上的输出井送到打印机

SPOOLing 技术在现代操作系统中还应用吗？

SPOOLing 技术并未过时,在现代计算机系统中仍有广泛应用:

- 打印系统:现代操作系统的打印队列管理

- 邮件系统:电子邮件的发送和接收过程

- 后台作业处理:批处理任务的调度和执行

- 数据库系统:事务处理和日志管理

5、传统分时系统在今天的意义

- 多用户支持:现代操作系统继承了分时系统的多用户并发访问能力

- 资源共享:分时思想是云计算、虚拟化等现代技术的基础

- 交互式计算:分时系统建立的交互式计算模

式仍是现代系统的核心特性

- 公平调度:分时系统的时间片轮转调度思想仍被广泛应用

6、操作系统的主要作用和典型特征

1)主要作用:

- 管理计算机硬件和软件资源

- 为应用程序提供统一的服务接口

- 实现人机交互,提供用户界面

- 提高系统资源利用率

- 保护系统安全和数据完整性

2) 典型特征(与其他软件相比):

- 并发性:能够同时运行多个程序

- 共享性:多个用户/程序共享系统资源

- 虚拟性:将物理资源抽象为逻辑资源

- 异步性:程序执行的结果与时间有关

- 持久性:操作系统常驻内存

- 特权性:拥有对硬件的直接控制权

- 复杂性:结构复杂,功能丰富

PTT2 运行环境和运行机制

1、应用程序是如何与操作系统交互的？

应用程序通过系统调用与操作系统交互。系统调用提供了应用程序访问操作系统服务的接口,例如文件操作、进程管理和内存管理等。

2、怎样理解“操作系统是由中断/异常/事件驱动的”这句话？

这句话的意思是操作系统的运行依赖于中断、异常和事件的触发。中断和异常是硬件或软件产生的信号,通知操作系统需要处理的事件。操作系统通过响应这些信号来管理系统资源和执行任务。

3、中断/异常的来源有什么不同？处理方式是一样的吗？

中断通常由外部设备 (如键盘、鼠标、网络接口等) 产生,而异常常由 CPU 在执行指令时检测到的错误 (如除零错误、非法指令等) 产生。处理方式有所不同,中断处理程序通常较为简单,主要负责响应外部设备的请求,而异常常处理程序则需要更复杂的错误处理机制。

4、回顾一下：ICS 对异常的描述及分类

ICS 将异常分为四类：陷入 (Trap)、故障 (Fault)、终止 (Abort) 和中断 (Interrupt)。陷入是由用户程序主动发起的系统调用,故障是可恢复的错误,终止是不可恢复的错误,中断是由外部设备发起的请求。

5、中断/异常处理流程中,哪些工作是硬件 (体系结构) 负责的？哪些工作是软件 (操作系统) 负责的？

硬件负责检测中断/异常、保存当前的处理器状态、查找中断向量表并跳转到相应的中断/异常处理程序。软件负责具体的中断/异常处理逻辑,包括错误处理、资源管理和恢复系统状态等。

6、从中断响应 (硬件) 到中断处理程序 (软件) 执行结束,计算机系统经过了哪些流程？

计算机系统首先由硬件检测到中断信号,保存当前处理器状态,查找中断向量表并跳转到中断处理程序。中断处理程序执行相应的处理逻辑,处理完成后恢复处理器状态,返回到中断前的执行点继续执行。

中断响应流程

1.设备发中断信号（e.g.键盘）

- CPU 处理完当前指令后检测到中断,判断出中断来源并向相关设备发确认信号。

2.硬件保存现场（context 上下文，主要指 PSW 和 PC）

PSW——program status word

程序状态字

PC——program counter

程序计数器

处理器状态被切换到内核态

3.根据中断码查中断向量表（操作系统准备好）-获得与该中断相关的处理程序的入口地址,并将 PC 设置成该地址,新的指令周期开始时, CPU 控制转移到中断处理程序。

4.把中断处理程序入口地址等推送到相应的寄存器

5.执行中断处理程序（软件）

- 保存相关寄存器信息

- 分析中断/异常的具体原因

- 执行对应的处理功能

- 恢复现场,返回被事件打断的程序

中断处理结束时, CPU 检测到中断返回指令,从系统栈中恢复被中断程序的上下文环境, CPU 状态恢复成原来的状态, PSW 和 PC 恢复成中断前的值, CPU 开始一个新的指令周期 (硬件)

7、操作系统初始化与中断/异常有哪些关联？

操作系统初始化时会设置中断向量表、初始化中断控制器、注册中断/异常处理程序等。中断/异常机制是操作系统正常运行的重要保障。

8、什么是软件异常？它是如何工作的？

软件异常是由软件引发的异常情况,例如非法内存访问、除零错误等。软件异常通过硬件检测并触发相应的异常处理程序,操作系统负责处理这些异常并采取相应的措施,如终止进程、生成错误报告等。

9、X86 有哪些控制和状态寄存器？所起的作用是什么？

X86 处理器有多个控制和状态寄存器,包括 CR0-CR4 (控制寄存器)、EFLAGS (状态寄存器)、GDTR/IDTR (全局/中断描述符寄存器) 等。控制寄存器用于控制处理器的工作模式,状态寄存器保存处理器的状态标志,描述符寄存器用于存储全局和中断描述符表的地址。

10、X86 在 PentiumII 300 之后提供了 sysenter/sysexit 指令,为什么？与 int 0x80/iret 有什么不同？X86-64 提供的系统调用指令是什么？

sysenter/sysexit 指令提供了更高效的系统调用机制,减少了系统调用的开销。与 int 0x80/iret 相比,sysenter/sysexit 指令不需要保存和恢复中断标志,减少了上下文切换的开销。X86-64 提供的系统调用指令是 syscall/sysret。

11、关于基于 x86 体系结构的 Linux 的系统调用实现:

1) 系统调用入口程序 system_call()与中断描述符表是什么关系？与系统调用表是什么关系？ system_call()是系统调用的入口程序,通过中断描述符表 (IDT) 中的中断向量指向。系统调用表 (sys_call_table) 存储了所有系统调用的地址, system_call()根据系统调用号查找并调用相应的系统调用处理程序。

2) 系统调用处理结束后,处理器转去执行哪个模块？

系统调用处理结束后,处理器会返回到用户态,

继续执行被中断的用户程序。

12、系统调用与函数/过程调用的区别是什么？系统调用与 C 函数调用的区别？系统调用与 API 的关系？

系统调用是操作系统提供的接口,用于应用程序请求操作系统服务。函数/过程调用是程序内部的调用机制。系统调用与 C 函数调用的区别在于系统调用需要从用户态切换到内核态,而 C 函数调用在用户态内执行。API (应用程序编程接口) 是应用程序与操作系统或库函数之间的接口,系统调用是 API 的一部分,提供底层操作系统服务。

PTT3 进程线程概念

1、怎样理解“进程是对 CPU 的抽象”这句话？

物理 CPU 只有一个 (或有限个),但通过多道程序设计和操作系统的进程调度,可以让多个程序在宏观上“同时”运行。操作系统为每个运行的程序创建一个进程,并管理它们对 CPU 的使用 (分时复用)。这使得每个进程都感觉自己仿佛独占了一个 CPU (或一个虚拟 CPU) 来执行自己的指令序列。因此,进程机制将一个或多个物理 CPU 虚拟化成了多个虚拟 CPU,供多个程序并发执行,这是对 CPU 计算能力的抽象。

2、何谓进程映像？进程有实体吗？在哪里？

进程映像 (Process Image) 是指进程在执行时的完整状态描述,是进程实体的静态体现。它包括:

程序代码 (Code Segment): 进程要执行的指令。

程序数据 (Data Segment): 程序使用的全局变量、静态变量等。

程序堆栈 (Stack): 用于函数调用、局部变量存储。

堆 (Heap): 动态分配内存的区域。

进程控制块 (PCB): 包含进程的所有管理信息 (状态、ID、寄存器值、资源列表等)。

进程是有实体的。它的实体就是进程映像所包含的内存区域 (代码、数据、堆栈、堆) 以及在操作系统内核中的数据结构 (PCB) - 这些实体主要存在于内存中 (代码、数据、堆栈、堆) 和操作系统内核空间 (PCB 及其相关数据结构) 。当进程被挂起时,部分映像可能被交换到磁盘 (交换空间) 上。

3、怎样描述进程？一个进程都有什么 (组成要素) ？

描述一个进程主要通过其**进程控制块 (PCB)**。PCB 是操作系统感知进程存在的唯一标志,包含了描述和控制进程运行所需的所有信息。

一个进程的组成要素 (即进程映像) 包括:

a.程序代码

b.数据集合 (全局变量、静态变量、动态分配的内存)

c.执行上下文 (CPU 寄存器值、程序计数器 PC、程序状态字 PSW、栈指针等)

d.进程控制块 (PCB) (包含进程标识符、状态、优先级、资源列表等)

4、创建进程主要完成哪些工作？

创建一个进程 (例如通过 fork()或 CreateProcess) 主要包括:

a.分配进程标识符 (PID): 给新进程一个唯一的 ID。

b.创建和初始化进程控制块 (PCB): 分配 PCB

结构,并填入初始信息 (如 PID、父进程 ID、初始状态设为 New 或 Ready、优先级等)。

c.分配地址空间: 为进程分配独立的虚拟内存空间 (可能通过复制父进程空间或加载新程序)。

d.加载程序和数据: 将可执行文件的代码和数据加载到进程的地址空间中 (exec 的工作)。

e.初始化执行上下文: 设置 PC 指向程序入口,初始化栈指针和寄存器。

f.分配资源: 分配进程所需的其它资源 (如文件描述符,继承自父进程或新创建)。

g.状态设置与调度: 将进程状态设置为就绪态 (Ready),并将其链入就绪队列,等待调度器分配 CPU。

5、进程的生命周期内都会经历哪些变化？怎样表示这些变化？

进程在其生命周期中会经历状态的转换。基本的状态包括:

创建态 (New): 进程正在被创建。

就绪态 (Ready): 具备运行条件,等待 CPU。

运行态 (Running): 正在 CPU 上执行。

等待态/阻塞态 (Waiting/Blocked): 等待某个事件 (如 I/O 完成) 而暂停执行。

终止态 (Terminated): 进程执行完毕或被终止,等待系统回收资源。

这些变化通常用“进程状态转换图”来表示,图中的节点代表状态,有向边代表状态之间的转换及其触发条件 (如调度、等待事件、事件完成等)。

6、进程有哪些状态？进程状态之间的转换 (条件？操作？)

基本状态: 运行态 (Running)、就绪态 (Ready)、等待态 (Waiting/Blocked)。

其他状态: 创建态 (New)、终止态 (Terminated)。还可能引入挂起态 (Suspended Ready, Suspended Blocked)。

常见转换及条件/操作:

New -> Ready: OS 完成进程创建的必要工作,资源基本到位,允许参与调度。

Ready -> Running: 进程被调度器 (Scheduler) 选中,获得 CPU 使用权。操作: 上下文切换,恢复进程现场。

Running -> Ready: 时间片用完;或被更高优先级的进程抢占。操作: 上下文切换,保存进程现场。

Running -> Waiting: 进程请求 I/O 操作或等待某一资源/事件。操作: 进程主动调用阻塞原语 (e.g., wait()), 保存现场,移入等待队列。

Waiting -> Ready: 进程等待的事件发生或资源可用 (如 I/O 完成)。操作: 中断处理程序或相关内核线程执行唤醒原语 (e.g., wakeup()), 将进程移入就绪队列。

Running -> Terminated: 进程正常执行完毕或出错退出。操作: 进程调用退出原语 (e.g., exit()), 进入终止态。

Terminated -> Gone: OS 回收进程所占资源 (PCB、内存等)。

7、进程状态转换的发生,是否一定导致另一个转换发生？

是的,通常是这样。操作系统是一个动态系统,进程状态转换往往是相互关联的。

例如,一个进程从 Running -> Waiting,会释放 CPU,这使得调度器可以选择另一个处于

Ready 状态的进程，使其发生 Ready -> Running 的转换。

一个进程从 Waiting -> Ready (如 I/O 完成)，它进入就绪队列，可能在未来某个时刻引发 Ready -> Running 的转换 (当它被调度时)。一个进程 Running -> Terminated，会释放它占有的资源，这可能导致另一个 Waiting (等待该资源) 的进程变为 Ready。

8、操作系统给进程提供内存空间，该空间的地址是虚拟地址还是物理地址？为什么？

操作系统提供给进程的地址空间是虚拟地址空间 (Virtual Address Space)。

原因：

1)隔离与保护：每个进程拥有独立的虚拟地址空间，一个进程无法直接访问另一个进程的内存，提供了安全保护。

2)地址空间扩展：虚拟地址空间可以大于物理内存，借助内存管理单元(MMU)和磁盘交换空间，给进程提供更大的可用地址范围。

3)内存管理简化：操作系统可以更灵活地管理物理内存，例如将非连续的物理内存页映射到连续的虚拟地址空间，简化了内存分配和程序加载。

4)程序加载和链接简化：程序可以在编译链接时确定其在虚拟地址空间的布局，而无需关心实际加载到物理内存的哪个位置。

9、操作系统如何描述进程的地址空间？

操作系统内核通常使用特定的数据结构来描述进程的地址空间。例如：

在 Linux 中，使用 mm_struct 结构来表示一个进程的整个地址空间。mm_struct 内部包含一个 vm_area_struct (VMA) 的链表或树，每个 VMA 描述了虚拟地址空间中的一个连续区域 (段)，包括其起始地址、访问权限 (读/写/执行)、映射的文件 (如果有) 等信息。

通过 页表 (Page Tables) 或段表 (Segment Tables) 将虚拟地址映射到物理地址。这些表由硬件 (MMU) 使用，操作系统负责维护。

可以通过 cat /proc/<PID>/maps 命令查看一个进程的虚拟地址空间布局和 VMA 信息。需要将 <PID> 替换为实际的进程 ID (可通过 ps 命令查找)，否则会提示文件或目录不存在。

10、为什么有了进程后又引入线程？

引入线程主要是为了解决进程的以下不足：

1)并发应用需求：许多应用内部包含多个并发执行的任务 (如 Web 服务器处理多个请求，GUI 程序响应用户输入同时后台处理)。用多进程实现这些任务，开销较大且通信复杂。

2)开销问题：创建进程、撤销进程、以及在进程间切换 (上下文切换) 都需要较大的时间和系统资源开销。线程作为“轻量级进程”，创建、销毁和切换的开销小得多。

3)通信效率：同一进程内的线程共享地址空间和大部分资源，它们之间的通信 (通过共享内存) 非常高效，无需内核介入。进程间通信 (IPC) 通常需要内核的协调，更复杂且效率较低。

4)性能提升：在多核处理器上，同一进程的多个线程可以真正并行执行在不同的核心上，提高应用程序的吞吐量。

11、怎样实现线程机制？为什么有各种支持线程的方式？

线程机制主要有三种实现方式：

1)用户级线程 (User-Level Threads, ULT)：线程的管理 (创建、调度、同步) 完全在用户空间中由一个线程库来完成。内核对线程无感知，只管理进程。

优点：切换快 (不需内核模式)，可自定义调度算法，可移植性好。

缺点：一个线程阻塞 (如系统调用)，整个进程会阻塞；无法利用多核并行。

2)核心级线程 (Kernel-Level Threads, KLT)：线程的管理由操作系统内核完成。内核知道每个线程的存在，并进行调度。

优点：一个线程阻塞不影响其他线程；可以利用多核并行。

缺点：线程创建、销毁、切换需要进入内核，开销比 ULT 大。

3)混合实现 (Hybrid Implementation)：结合了 ULT 和 KLT。内核管理 KLT，用户空间线程库将多个 ULT 映射到一个或多个 KLT 上。

目标：兼具两者的优点，但实现复杂。

*存在多种方式的原因：是在性能、并发能力、实现复杂度、系统资源消耗之间进行权衡的结果。

ULT 优先考虑低开销和灵活性。

KLT 优先考虑真正的并发和对阻塞系统调用的处理。

混合模型试图找到一个平衡点。

12、线程包 pthreads 中相关的函数的功能？

Pthreads (POSIX Threads) 是一个线程 API 标准，提供了一系列函数来管理线程：

pthread_create(): 创建一个新的线程。

pthread_exit(): 终止调用该函数的线程。

pthread_join(): 等待指定的线程终止。

pthread_yield(): 主动让出 CPU，让其他线程运行。

pthread_self(): 获取调用线程自身的线程 ID。

pthread_mutex_init(),pthread_mutex_lock(), pthread_mutex_unlock(),

pthread_mutex_destroy(): 互斥锁相关操作，用于保护临界区，实现线程互斥。

pthread_cond_init(),pthread_cond_wait(), pthread_cond_signal(),pthread_cond_broadcast(), pthread_cond_destroy(): 条件变量相关操作，用于线程间的同步 (等待某个条件满足)。

13、中断/异常机制与进程线程模型的关联？

中断和异常是操作系统得以实现进程/线程调度和管理的关键机制。

1)上下文切换触发：时钟中断 (Timer Interrupt) 使得操作系统可以剥夺当前运行进程/线程的 CPU 使用权 (时间片用完)，进行调度，切换到其他就绪的进程/线程。这是实现分时复用的基础。

2)状态转换：I/O 完成中断会通知操作系统，操作系统可以将等待该 I/O 的进程/线程从等待态转换为就绪态。

3)系统调用：进程通过执行特定的指令 (如 syscall 或 int 0x80) 产生异常 (陷阱 Trap)，主动陷入内核态，请求操作系统服务 (如创建进程、读写文件、阻塞等待)。内核处理完请求后，可能会进行调度。

4)错误处理：异常 (如除零、缺页故障 Page Fault) 也需要内核介入处理。缺页故障处理是虚拟内存管理的核心部分，可能导致进程阻塞 (等待页面从磁盘调入)。

5)保存与恢复现场：发生中断/异常时，硬件和操作系统内核合作，必须保存当前进程/线程的执行上下文 (寄存器、PC、状态等)，处理事件后，再恢复某个进程/线程 (可能是同一个，也可能是不同的) 的上下文继续执行。

14、机制和策略分离的原则在进程线程模型中的体现？

有点像操作系统提供的 API 和调用这些 API 的策略之间的关系。

机制与策略分离是操作系统设计的重要原则，意指提供实现某种功能的基础能力 (机制)，与决定何时、如何使用这些能力的决策逻辑 (策略) 分开。

进程/线程状态管理 (机制) vs. 调度算法 (策略)：操作系统提供了进程/线程状态 (就绪、运行、等待等) 以及在它们之间转换的机制 (如阻塞/唤醒原语、上下文切换)。但是，选择哪个就绪进程/线程投入运行，则是调度算法 (策略) 决定的 (如 FIFO、轮转、优先级调度等)。

挂起/激活 (机制) vs. 负载调节 (策略)：操作系统提供将进程换出到磁盘 (挂起) 和换回内存 (激活) 的机制。但是，决定何时挂起哪个进程 (例如，为了降低内存压力或提高系统吞吐量)，则是系统负载调节策略的一部分。

线程实现 (机制) vs. 应用并发模型 (策略)：用户级线程库提供创建和管理线程的机制。应用程序如何利用这些线程来构建并发逻辑 (例如，线程池大小、任务分配方式) 则是应用层面的策略。

15、协程是什么？为什么引入协程？协程怎么用？

1)是什么：协程是一种比线程更轻量级的用户态并发 (或协作式多任务) 实现方式。它们可以在特定点上暂停执行，并在稍后从同一点恢复执行的计算过程。协程之间的切换由程序员 (或协程库/语言运行时) 显式控制，通常发生在用户态，不需要内核介入。

2)为什么引入：

a.极低的切换开销：协程切换完全在用户态进行，避免了内核态和用户态之间的切换以及内核调度，开销远小于线程切换。

b.高并发能力：单个线程可以管理成千上万个协程，特别适合处理大量并发连接 (如网络服务器) 或 I/O 密集型任务，能有效减少线程数量和内存消耗。

c.简化异步编程：允许使用看似同步的代码风格来编写异步逻辑 (例如，使用 async/await 关键字)，从而避免回调地狱 (Callback Hell)。

回调地狱是指在传统异步编程中，当一个操作依赖于另一个异步操作的结果时，需要将其后续操作放在前一个操作的回调函数中，如果存在多层依赖，就会形成层层嵌套的回调函数结构，导致代码难以阅读、理解和维护。

3)怎么用：通常通过编程语言或第三方库提供的支持来使用。

语言原生支持：如 Python (async/await), Go (goroutine), C++20 (co_await, co_yield, co_return), Rust (async/await)。

库支持：通过特定的协程库在不支持原生协程的语言中使用。

用法：开发者定义协程函数，在需要等待的操作 (通常是 I/O) 前使用特定关键字 (如 await

或 yield) 暂停当前协程，让出执行权给其他协程或事件循环，当操作完成后，协程从暂停点恢复执行。

PPT4 进程线程调度

When&How

1、为什么操作系统需要“调度”？CPU 只有一个时，怎么同时跑多个程序？

控制、协调进程对 CPU 的竞争。

按一定的调度算法从就绪队列中选择一个进程，把 CPU 的使用权交给被选中的进程。

2、调度这件事什么时候做？做的理由有哪些？

a.调度时机 (When): 操作系统需要在特定事件发生时决定哪个进程接下来应该占用 CPU。这些时机主要包括：

1)进程创建：当一个新进程被创建时 (例如，通过 fork()系统调用)，需要决定是运行父进程还是子进程，或者其他进程。

2)进程终止：当一个进程执行完毕或被终止时 (例如，调用 exit())，它占用的 CPU 必须分配给其他就绪的进程。

3)进程阻塞：当一个进程因等待某个事件 (如 I/O 操作完成、等待信号量、等待用户输入等) 而无法继续执行时，它会进入阻塞状态，此时调度器需要选择另一个进程来运行。

4)I/O 中断发生：当一个 I/O 操作完成，产生中断时，原先等待该 I/O 的进程可能会从阻塞态变为就绪态。这时，调度器可能需要重新评估，决定是继续运行当前进程，还是切换到刚刚变为就绪的、可能优先级更高的进程。

5)时钟中断发生：在分时系统中，为了防止某个进程长时间独占 CPU，操作系统会设置一个定时器。当定时器中断发生时，当前运行进程的时间片 (Time Slice/Quantum) 可能已用完，调度器会介入，决定是继续运行该进程 (如果时间片未用完或没有其他就绪进程)，还是切换到另一个就绪进程 (抢占式调度)。

b.调度的理由 (Why): 调度的根本目的是有效、公平地管理和分配有限的 CPU 资源给多个并发执行的进程。具体理由包括：

1)提高 CPU 利用率：尽量让 CPU 保持忙碌状态，减少空闲时间。当一个进程等待 I/O 时，可以让其他就绪进程使用 CPU。

2)提高系统吞吐量：单位时间内完成的进程数量。好的调度算法可以在满足其他目标的同时，尽可能多地完成任务。

3)减少周转时间：指一个进程从提交到完成所花费的总时间 (等待进入内存、在就绪队列等待、CPU 执行、I/O 执行的总和)。

4)减少等待时间：指进程在就绪队列中等待 CPU 所花费的总时间。

5)减少响应时间：对于交互式系统尤其重要，指从用户发出请求到系统首次产生响应 (而非完成任务) 所花费的时间。

6)确保公平性：保证每个进程都能获得合理的 CPU 时间份额，防止某些进程被饿死。

7)满足实时性要求：对于实时系统，调度必须保证关键任务在它们的截止时间 (Deadline) 之前完成。

3、如果没有可被调度的进程，系统做什么呢？

如果当前没有用户进程或系统核心任务进程处于就绪状态 (Ready State)，CPU 不能完全停

止。操作系统通常会执行一个特殊的空闲进程 (Idle Process) 或称为系统空闲任务 (System Idle Task)。

作用：这个进程拥有最低的优先级。当没有其他任何事情可做时，调度器就会选择它来运行。行为：

它通常执行一个无限循环。

在这个循环中，它可以执行一些低优先级的系统维护任务。

更重要的是，在许多架构上 (如 x86)，它可以执行一个特殊的指令 (如 HLT - Halt)，使 CPU 进入低功耗状态，直到下一个中断 (如时钟中断、I/O 中断) 唤醒 CPU。这有助于节能和降低温度。

目的：确保 CPU 总是有事可做 (即使是“等待”)，并提供一个合法的状态供调度器切换，同时优化能源使用。

4、上下文切换的过程？有哪些开销？

上下文切换 (Context Switch) 是指操作系统保存当前正在运行进程的状态 (上下文)，并加载另一个进程的状态，以便让后者开始或继续运行的过程。这是实现多任务处理的基础。

a.过程：

1)中断/系统调用触发：调度发生 (如时间片用完、进程阻塞等)。

2)保存当前进程上下文：保存程序计数器 (Program Counter, PC) 和其他 CPU 寄存器 (通用寄存器、状态寄存器等) 的值。这些值通常保存在该进程的进程控制块 (Process Control Block, PCB) 中。

保存当前进程的栈指针。

更新新进程状态 (例如，从 "Running" 变为 "Ready" 或 "Blocked")。

可能需要保存内存管理相关信息 (如页表基址寄存器)。

3)执行调度算法：操作系统调度器代码运行，根据调度策略选择下一个要运行的进程。

4)加载新进程上下文：从选定进程的 PCB 中恢复其状态。

加载新进程的寄存器计数器和 CPU 寄存器。恢复新进程的栈指针。

更新新进程的状态 (通常是从 "Ready" 变为 "Running")。

恢复内存管理信息 (可能需要刷新 TLB - Translation Lookaside Buffer)。

5)跳转执行：CPU 跳转到新进程被中断时的下一条指令地址 (或其入口点，如果是首次运行) 开始执行。

b.开销：上下文切换本身并不执行任何有用的用户工作，它是一种纯粹的开销。开销主要包括：

1)直接开销：保存和加载寄存器：CPU 需要时间来读写寄存器和 PCB。

执行调度器代码：选择下一个进程也需要 CPU 时间。

更新 PCB 和其他数据结构：维护进程队列等操作需要时间。

MMU 操作：可能需要加载新的页表基址，这可能导致 TLB 被刷新 (TLB flush)，增加后续内存访问的延迟。

2)间接开销：缓存污染 (Cache Pollution)：当新进程开始运

行时，CPU 缓存 (L1, L2, L3 Cache) 中很可能包含的是前一个进程的数据和指令。新进程运行时会发生大量的缓存未命中 (Cache Miss)，需要从内存中重新加载数据，这会显著降低执行速度，直到新进程的“工作集” (Working Set) 被加载到缓存中。这是上下文切换最主要的性能影响之一。

CPU 流水线冲刷：切换可能导致 CPU 的指令流水线被清空和重建。

最佳实践：频繁的上下文切换会显著降低系统整体性能。因此，调度算法和系统设计 (如时间片大小的选择) 需要在响应时间和系统吞吐量/效率之间找到平衡。

5、关于调度算法，我们都关心什么？

1)CPU 利用率：CPU 处于忙碌状态的时间百分比。越高越好，但 100%可能意味着没有冗余，响应性可能变差。

2)系统吞吐量 (Throughput)：单位时间内完成的进程 (或作业) 数量。越高越好。

3)周转时间：从进程提交到完成的总时间。越短越好 (平均周转时间、最差周转时间)。

4)等待时间：进程在就绪队列中等待 CPU 的总时间。越短越好 (平均等待时间、最差等待时间)。

5)响应时间：从提交请求到产生第一个响应的时

间 (交互式系统关键)。越短且越稳定越好。

6)公平性：每个进程获得合理的 CPU 份额，防止饿死。

7)可预测性：对于实时系统，执行时间的可预测性比平均性能更重要。

8)满足截止时间：对于实时系统，这是硬性或软性截止。

9)优先级处理：系统能否有效处理不同优先级的进程。

10)资源平衡：尽量保持所有资源 (CPU, I/O 设备) 都处于忙碌状态。

6、不同类型的操作系统都适用同一种调度算法吗？

不适用。不同类型的操作系统有着不同的设计目标和用户需求，因此需要采用不同的调度策略。

1)批处理系统 (Batch Systems)：目标：最大化吞吐量和 CPU 利用率，减少平均周转时间。用户通常不直接与系统交互。

适用算法：先来先服务 (FCFS)、最短作业优先 (SJF)、最高响应比优先 (HRRN)。公平性和响应时间相对不重要。

2)交互式系统：目标：最小化响应时间，提供良好的用户体验，兼顾公平性。

适用算法：轮转法 (Round Robin, RR)、优先级调度、多级队列调度 (Multi-level Queue)、多级反馈队列调度 (Multi-level Feedback Queue, MLFQ)。

3)实时系统 (Real-Time Systems, RTOS)：目标：满足任务的截止时间要求，可预测性至关重要。分为硬实时 (必须满足) 和软实时 (尽量满足)。

适用算法：速率单调调度 (Rate Monotonic Scheduling, RMS - 用于静态优先级)、最早截止时间优先 (Earliest Deadline First, EDF - 用于动态优先级)、优先级调度 (配合精确的优先级分配)。

7、对于一个调度算法，应该追求什么样的目标？

一个调度算法应该追求的目标组合取决于具体的系统类型和应用场景。通常需要在多个（有时是相互冲突的）目标之间进行权衡（Trade-off）。

1)通用目标（All Systems):

公平性: 防止饿死。

策略强制: 确保系统设定的策略（如优先级）得到执行。

平衡: 保持系统的各个部分都处于活动状态（例如，CPU 密集型 and I/O 密集型进程交替运行）。

2)批处理系统目标:

高吞吐量

低周转时间

高 CPU 利用率

3)交互式系统目标:

快速响应时间

低响应时间方差（稳定性）

满足用户期望（感觉流畅）

4)实时系统目标:

满足截止时间

高可预测性

最佳实践: 没有“万能”的调度算法。选择或设计算法时，必须明确系统的主要目标，并接受在其他方面可能存在的不足。例如，追求极低响应时间可能会牺牲一些吞吐量。

8、选进程时都考虑了哪些点？单一因素还是多因素？

选择一个要运行的进程时，调度算法可能考虑单一因素或多个因素。

单一因素:

FCFS: 只考虑进程到达就绪队列的时间。

SJF（非抢占式）: 只考虑预估的下一个 CPU 脉冲（burst）长度。

简单优先级调度: 只考虑静态分配的优先级。

多因素: 现代操作系统和更复杂的调度算法通常是多因素的。

优先级调度（带老化）: 考虑静态优先级，但也考虑进程等待时间（老化机制，aging），提高等待过久进程的优先级以防饿死。

RR: 考虑到达时间和时间片轮转。

MLFQ: 考虑优先级、进程行为（CPU 密集型 vs I/O 密集型）、等待时间等，动态调整进程在不同队列间的移动。

HRRN: 考虑等待时间（W）和服务时间/脉冲长度（S），计算响应比（W+S/S）。

Linux CFS: 考虑进程的虚拟运行时间（virtual runtime），旨在给每个进程公平的 CPU 时间比例。它间接考虑了进程的等待时间和已运行时间。

Windows 调度: 考虑基础优先级、动态优先级提升（如完成 I/O、处于前台窗口）、时间片消耗情况等。

结论: 简单算法可能只关注单一因素，但为了在复杂环境中平衡多个目标（如响应时间、公平性、吞吐量），现代通用操作系统广泛使用多因素调度算法。

What

1、适用批处理系统的调度算法有哪些？

主要目标是效率（吞吐量、CPU 利用率）和整体完成时间（周转时间）。

1)先来先服务(First-Come, First-Served, FCFS): 实现: 按进程到达就绪队列的顺序进行调度。

使用 FIFO 队列。

优点: 简单，易于实现，公平（按到达顺序）。

缺点: 平均等待时间可能很长，尤其当短进程排在长进程之后时（护航效应 Convoy Effect）。不适合交互式系统。是非抢占式。

2)最短作业优先（Shortest Job First, SJF):

实现: 选择预计 CPU 执行时间（下一个 CPU burst）最短的进程。可以是抢占式（SRTF - Shortest Remaining Time First）或非抢占式。

优点: 理论上具有最优的平均等待时间和平均周转时间。

缺点:

需要预测下一个 CPU burst 长度，这很难精确做到（通常基于历史数据估计）。

可能导致长作业饿死（Starvation），即长时间得不到 CPU。

非抢占式 SJF 不适合交互系统。抢占式 SRTF 开销较大。

3)最高响应比优先(Highest Response Ratio Next, HRRN):

实现: 非抢占式。计算每个进程的响应比 $R = (\text{等待时间 } W + \text{服务时间 } S) / \text{服务时间 } S$ ，选择 R 最高的进程。

优点: 结合了 FCFS 和 SJF 的优点。短作业容易被选中（S 小，R 大）。同时，等待时间长的进程其响应比也会增加，避免了饿死。

缺点: 仍需要预测服务时间 S。计算响应比有额外开销。

2、适用交互式系统的调度算法有哪些？

主要目标是提供快速响应和用户满意度。

1)轮转法（Round Robin, RR):

实现: 类似于 FCFS，但增加了时间片和抢占。每个进程被分配一个固定的时间片（Quantum），运行时间超出时间片后会被强制切换（抢占），放回就绪队列尾部。

优点: 公平，响应时间相对较短（特别是对于短请求），简单。

缺点:

性能对时间片大小非常敏感。太小则上下文切换频繁，开销大；太大则退化为 FCFS，响应时间变长。

平均周转时间通常比 SJF 长。

没有考虑优先级。

最佳实践: 时间片大小通常选择比平均交互响应所需时间稍长，但足够短以保证多个交互进程能快速轮转。

2)优先级调度（Priority Scheduling):

实现: 为每个进程分配一个优先级，调度器总是选择就绪队列中优先级最高的进程。可以是抢占式或非抢占式。

优点: 可以明确区分重要任务和次要任务。

缺点:

可能导致低优先级进程饿死。

优先级的确定可能是个问题（静态 vs 动态）。改进:

老化（Aging): 随时间增加等待进程的优先级。动态优先级: 根据进程行为（如 I/O 等待）调整优先级。

3)多级队列调度（Multi-level Queue Scheduling):

实现: 将就绪队列划分为多个独立的队列，每

个队列有自己的优先级和调度算法（如：前台交互队列用 RR，后台批处理队列用 FCFS）。进程被永久分配到一个队列。

优点: 可以为不同类型的进程应用不同的调度策略。开销较低。

缺点: 缺乏灵活性，进程无法在队列间移动。不适合交互式系统。是非抢占式。

4)多级反馈队列调度（Multi-level Feedback Queue, MLFQ):

实现: 允许多个队列，并且进程可以在队列之间移动。这是目前通用操作系统中最常用的调度方法之一。

规则示例:

新进程进入最高优先级队列。

如果在时间片内完成，离开系统；如果用完时间片，则降级到下一个较低优先级队列。

在较低优先级队列中等待时间过长的进程可以被提升到较高优先级队列（防止饿死，即老化）。

I/O 密集型进程（经常阻塞放弃 CPU）通常会停留在较高优先级队列，保证响应性。CPU 密集型进程会逐渐下降到较低优先级队列。

优点: 非常灵活，自适应。能同时照顾到交互式和批处理式需求，兼顾响应时间、周转时间和公平性。

缺点: 设计和调优复杂（队列数量、各队列调度算法、时间片大小、升级降级策略）。

3、适用实时系统的调度算法有哪些？

主要目标是满足时间约束（截止时间）。

1)速率单调调度（Rate Monotonic Scheduling, RMS):

类型: 静态优先级，抢占式。

原理: 周期性任务的优先级与其执行频率（速率）成正比。周期越短（频率越高），优先级越高。

优点: 简单，易于实现，是最佳的静态优先级调度算法（如果任务集可调度，RMS 就能找到调度方案）。可进行理论上的可调度性分析（例如，利用率测试）。

缺点: 只适用于周期性任务；对任务特性有较强假设；CPU 利用率上限不如动态优先级算法高。

2)最早截止时间优先（Earliest Deadline First, EDF):

类型: 动态优先级，抢占式。

原理: 当前就绪任务中，绝对截止时间最早的任务拥有最高优先级。

优点: 理论上是最优的动态优先级调度算法。只要任务集的总 CPU 利用率不超过 100%，EDF 就能找到调度方案（对于可抢占、独立任务等理想情况）。可以处理周期性和非周期性任务。

缺点: 实现比 RMS 复杂；可能出现多米诺骨牌效应（一个任务错过截止时间可能导致后续任务也错过）；动态优先级变化导致上下文切换可能更频繁。

3)基于优先级的抢占式调度:

通用方法: 给实时任务分配高优先级，使用标准优先级调度器。可以通过精心设置优先级来模拟 RMS 或 EDF 的行为。常用于软实时系统或硬实时系统中与其他任务共存的情况。

需要确保高优先级任务能抢占低优先级任务，并且优先级反转问题得到处理（如使用优先级继承 Priority Inheritance 或优先级天花板

Priority Ceiling Protocol）。

4、怎样理解抢占式和非抢占式？

这是调度器根据何时进行调度的两种基本模式。

1)非抢占式调度:

定义: 一旦 CPU 分配给某个进程，该进程将一直运行，直到它主动放弃 CPU（完成任务、阻塞等待 I/O、或显式调用 yield）。调度器不能在进程运行中途强制剥夺其 CPU 使用权。

优点: 实现简单；上下文切换只在进程自愿放弃 CPU 时发生，开销相对较小；不会有并发访问内存数据结构的竞争问题（在单处理器上）。

缺点: 一个长时间运行或行为不当的进程可以独占 CPU，导致其他进程（特别是需要快速响应的交互式进程）长时间等待，响应性差；不适合分时和实时系统。

例子: 早期的 Windows（如 Windows 3.1），早期的 Mac OS，某些简单的嵌入式系统。

2)抢占式调度:

定义: 操作系统可以强制暂停当前正在运行的进程（即使它并未主动放弃 CPU），并将 CPU 分配给另一个进程。这种抢占通常发生在时钟中断（时间片用完）或更高优先级进程变为就绪时。

优点: 能够保证 CPU 在进程间公平分配；提供更好的响应时间；可以有效处理优先级，防止低优先级任务阻塞高优先级任务（前提是优先级设置合理）；是现代多任务操作系统的标准做法。

缺点: 实现更复杂；上下文切换更频繁，开销更大；需要处理内核数据结构的并发访问问题（需要锁或其他同步机制）。

例子: Unix/Linux, Windows NT 及后续版本, macOS，大多数现代 RTOS。

5、从哪几方面对调度算法进行比较？

主要从前面提到的性能指标/目标来进行比较和评估:

CPU 利用率: 哪个算法更能让 CPU 保持忙碌？吞吐量: 哪个算法单位时间内能完成更多任务？

周转时间（平均/最坏): 哪个算法下进程从提交到完成更快？

等待时间（平均/最坏): 哪个算法下进程在就绪队列中等待的时间更短？

响应时间（平均/方差): 哪个算法对交互式请求的响应更快、更稳定？

公平性: 哪个算法更能保证所有进程获得合理的 CPU 时间，避免饿死？

可预测性/满足截止时间: 对于实时系统，哪个算法更能保证任务按时完成？

算法开销: 算法本身的计算复杂度以及它导致的上下文切换频率和开销如何？

实现复杂度: 算法是否容易实现和调试？对参数的敏感性: 算法性能是否严重依赖于某些参数（如时间片大小、优先级设置）？

6、机制和策略分离的原则在调度算法中的应用

机制（Mechanism): 提供如何做（How）的基础能力或工具。在调度中，机制包括:

上下文切换的代码（保存/加载寄存器、切换页表）。

维护进程状态（就绪、运行、阻塞）和 PCB 的数据结构。

管理就绪队列（如链表、优先级队列、红黑树）。

时钟中断处理程序。

提供设置和读取进程优先级接口的。

进程挂起和唤醒的原子操作。

策略(Policy): 决定做什么（What）或何时做（When）。在调度中，策略是指具体的调度算法逻辑:

如何选择下一个运行的进程（FCFS 规则？SJF 规则？RR 规则？优先级规则？）。

时间片长度是多少？

优先级如何确定？是静态还是动态调整？如何调整？

何时进行抢占？

如何处理不同类型的进程（前台/后台，实时/普通）？

例子: 调度器的主循环可能是一个通用的框架（机制），它调用一个函数（策略）来选择下一个进程。不同的调度算法可以通过实现这个选择函数来插入。上下文切换函数是另一个独立的机制。优先级队列的实现（如使用堆或链表）是机制，而如何利用这个队列（是按优先级取还是按 FIFO 取）是策略。

7、实例操作系统的调度算法都是什么？

Linux:

主要调度器（针对普通进程): 完全公平调度器（Completely Fair Scheduler, CFS），自内核 2.6.23 起引入。

目标: 为所有运行中的任务提供尽可能公平的 CPU 时间份额。

机制: 不再基于固定时间片，而是维护每个任务的虚拟运行时间（vruntime）。总是选择 vruntime 最小的任务来运行。任务运行会增加其 vruntime。I/O 等待的任务 vruntime 增长慢，因此返回时更容易被选中。

实现: 使用红黑树来高效地找到 vruntime 最小的任务。

实时调度策略:

SCHED_FIFO: 静态优先级的先来先服务（非时间片轮转）。相同优先级的任务按到达顺序执行，直到阻塞、退出或被更高优先级抢占。

SCHED_RR: 静态优先级的轮转法。同 SCHED_FIFO，但增加了时间片，同一优先级任务轮流运行。

优先级: 实时任务优先级高于普通任务。普通任务也有优先级（nice 值），但 CFS 主要通过 vruntime 实现公平性，nice 值影响 vruntime 增长的速度。

Windows (NT 内核及以后):

采用基于优先级的抢占式调度算法，结合了多级反馈队列的思想。

优先级: 分为 32 个优先级。0 为系统空闲线程，1-15 为可变优先级类（Variable Priority Classes），16-31 为实时优先级类（Real-time Priority Classes）。内核线程可能使用更高的内部优先级。

动态调整:

对于可变优先级类，系统会动态提升线程的优先级（Priority Boost），例如:

当线程完成 I/O 操作时。

当等待事件/信号量被满足时。

前台窗口的线程优先级通常会被提升，以改善交互响应。

短时间片用完后，优先级可能会暂时降低。

长时间消耗 CPU 的线程其动态优先级会逐渐

衰减回基础优先级。

时间片: 不同优先级的线程可能有不同的时间片长度。前台进程的时间片通常比后台进程长且可变。

Quantum: Windows 中称为 Quantum，不是固定的，可以根据系统设置和前后台状态调整。

macOS:

基于 XNU 内核（混合了 Mach 微内核和 BSD Unix）。

调度也采用基于优先级的抢占式模型，具有多级反馈特性。

线程优先级分为几个主要波段（bands): 内核模式、系统高优先级、用户交互（UI 响应关键）、用户启动、后台任务等。

动态调整: 系统会根据线程的行为（如 CPU 使用情况、是否阻塞等待 I/O、是否与用户界面交互）动态调整其优先级，类似于 Windows。

也使用了类似 Mach 的线程调度原语和概念，例如时间片捐赠（Time-sharing donation）等机制来优化性能。

近年来引入了服务质量（Quality of Service, QoS）类的概念，让开发者可以指定任务的意图（如用户交互、后台数据处理、维护任务），系统据此进行更智能的资源（包括 CPU 调度）管理。

总结: 现代主流操作系统都使用抢占式、基于优先级的调度框架，并结合动态优先级调整、多级队列/反馈机制，以及针对公平性（如 Linux CFS）或服务质量（如 macOS QoS）的特定优化，以适应应用计算环境中复杂多变的需求。

9、内存管理概述

1、内存管理从简单到复杂的演化过程是什么？

宏观演化（内存使用方式）: 早期物理内存独占使用→多道程序设计/分时共享→虚拟内存（OS 和多个进程共存并隔离）。

微观演化（具体管理方案）: 单一连续区→固定分区→可变分区→页式→段式→段页式。（解决碎片问题，并逐渐从满足物理内存连续性向支持离散性、按逻辑模块分配过渡）。

2、怎样理解“地址空间（address space）是对内存的抽象”？

1)解耦物理内存: 每个进程拥有自己独立、私有的地址空间，程序发出的地址（逻辑地址）与实际存放的物理地址无关。

2)屏蔽底层复杂性: 底层物理内存可能是离散的、需要交换的、多次装载的，但地址空间将其抽象为对进程来说是连续的、独占的、一次性的假象。

3)核心目标: 实现内存的“虚拟化”，为进程提供透明性（无需关心物理位置）、效率和保护（进程间互不干扰）。

3、内存管理追求的目标有哪些？

虚拟化内存（透明性、效率、保护）。给进程分配内存（分配地址空间）。

往内存加载内容（映射进程地址空间到物理内存）。

存储保护（防止地址越界、控制读/写/执行权限）。管理共享的内存（如共享库、内存映射文件）。

最小化存储访问时间（结合 Cache 等存储体系）。

4、程序什么时候进入内存？

程序装载到内存才可以运行。将指令和数据绑

定到内存地址的时机分为四种：

编译时 (Compile time)

链接时 (Linking time)

装载时 (Load time)

运行时 (Run time) (现代 OS 常用，支持动态重定位)

在内存紧张时，也可能在需要调度运行 (交换换入) 或发生缺页时才将特定部分移入内存。

5、进程空间与物理内存空间怎么对应？

通过地址重定位 (地址转换/映射) 机制，将进程空间中的逻辑地址 (虚拟地址) 转换为物理内存中的物理地址 (绝对地址)。

1)静态地址重定位：程序加载到内存时，一次性由软件实现逻辑地址到物理地址的转换。

2)动态地址重定位：在进程执行过程中，逐条指令执行时通过硬件 (如内存管理单元 MMU + 重定位寄存器) 完成地址映射。

具体对应方式依赖于管理方案，如：利用【页号查页表】映射到【物理页框】，或利用【段号查段表】映射到【段起始物理地址】。

6、物理内存空闲空间怎么管理？有哪些数据结构 and 算法？

1)数据结构：

位图 (Bitmap)：每个分配单元对应一位 (0 表示空闲，1 表示占用)。

空闲区表/已分配区表：记录起始地址、长度、标志。

空闲块链表：隐式空闲链表、显式空闲链表、分离空闲链表。

2)分配算法：

首次适配 (First fit)：在空闲区表中找到第一个满足要求的。

下次适配 (Next fit)：从上次查找到的位置接着往后找。

最佳适配 (Best fit)：遍历全表，找能够满足要求的最小空闲区。

最差适配 (Worst fit)：总是分配满足要求的最大空闲区。

7、怎样“扩充”内存？

当程序大小超过物理内存或内存空间紧张时，采用以下技术在逻辑上“扩充”内存：

1)内存紧凑 (Memory Compaction/压缩技术)：在内存中移动进程，将所有小碎片 (外碎片) 合并为大空闲区。

2)覆盖技术 (Overlaying)：将程序按逻辑结构划分，不会同时执行的程序段分时共享同一块内存区域 (时间换空间，需程序员声明)。

3)交换技术 (Swapping)：将内存中暂时不用的进程移出到外存 (Swap 区)，将外存中需要运行的进程换入内存 (与调度器结合)。

4)虚存技术：基于局部性原理，允许进程的部分内容驻留在外存，其它留在外存，实现更大的逻辑地址空间。

8、Linux 的伙伴系统 (Buddy System) 的作用？

作用：是 Linux 底层采用的经典内存分配方案，用于高效分配和回收连续的物理内存页框。

原理机制：将可用内存按 2 的幂进行划分 (组成若干空闲块链表)。一种特殊的“分离适配”算法。

运行方式：当申请 s 大小空间时，一直将大块均分为两个相等的“伙伴”子块，直到分出大于等于 s 的最小块分配给进程；回收时，如果左右相

邻的“伙伴”块都空闲，则自动合并为更大的块，有效缓解了外碎片问题。

小测 (25 秋+26 春)

Part 3 操作系统面试

1. 机制与策略分离设计原则在计算机各类系统设计时是如何应用的？

核心理念：将“功能如何实现” (机制) 与“如何使用该功能” (策略) 解耦。

应用体现：

微内核设计：内核仅提供最基础的机制 (如进程间通信、内存映射)，而将资源调度、文件系统等策略移交给用户态服务程序。

图形系统：显示驱动提供画线、填充的机制，而 UI 框架决定窗口的布局和颜色策略。

网络协议栈：物理层提供数据传输机制，而应用层根据不同业务需求制定数据交互策略。

2. 机制与策略分离设计原则在操作系统设计时是如何应用的？

调度机制 vs. 调度策略：内核提供上下文切换、时钟中断处理等机制；而决定下一个运行哪个进程、运行多久，则由策略 (如 RR、CFS 算法) 决定。

存储机制 vs. 置换策略：内核提供页表维护、缺页中断响应机制；而当内存不足时选择牺牲哪个页面，则由页面置换策略 (如 LRU、FIFO) 决定。

好处：增强了系统的灵活性和可扩展性。开发者可以针对不同应用场景 (如服务器 vs. 嵌入式) 更换策略，而无需修改底层复杂的机制代码。

3. 以缓存机制设计为例，其中哪些是机制？哪些是策略？

机制 (Mechanism)：缓存单元的存储结构 (数据存放的地方)。

数据的查找逻辑 (如何匹配 Key)。

数据的读写接口 (如何存入或取出数据)。

策略 (Policy)：替换策略：缓存满时，是移除最久未使用的 (LRU) 还是最先进入的 (FIFO)？

写入策略：数据更新时，是立即写回磁盘 (Write-through) 还是延迟写回 (Write-back)？

预取策略：是否根据访问模式提前加载后续数据？

4. “All problems in computer science can be solved by another level of indirection.” 请举例说明对这句话的理解。

核心含义：通过引入一个“中间层”或“映射机制”，使系统上下两层解耦，从而解决复杂性和灵活性问题。

典型例子：

虚拟内存：在程序 (逻辑地址) 与物理内存之间增加页表。程序不再关心实际物理位置，实现了内存保护和地址空间隔离。

文件系统：在磁盘块与用户之间增加文件名/路径。用户通过名字访问数据，而无需记住数据存储在磁盘的哪个扇区。

DNS 域名系统：在 IP 地址与用户之间增加域名。即使服务器 IP 变化，用户依然可以通过不变的域名访问。

Part 3 操作系统运行环境和运行机制

1. 列举出 6 种中断或异常事件：类型、具体原因。

因。

系统调用 (System Call)：类型为异常 (意料之外的同步事件)。具体原因：用户程序主动执行 `ecall` 指令，请求内核提供服务 (如打开文件)。

缺页异常 (Page Fault)：类型为异常。具体原因：进程访问的虚拟地址在页表中不存在对应的物理页映射，或访问权限不匹配。

时钟中断 (Timer Interrupt)：类型为中断 (异步硬件事件)。具体原因：硬件计时器达到预设值，用于强制剥夺当前进程 CPU 占用权以进行调度。

非法指令 (Illegal Instruction)：类型为异常。具体原因：CPU 尝试执行一段未定义或当前特权级不允许执行的二进制指令。

外设中断 (I/O Interrupt)：类型为中断。具体原因：外部设备 (如磁盘完成读写、网卡收到数据包) 向 CPU 发出信号请求处理。

断点异常 (Breakpoint)：类型为异常。具体原因：执行了调试指令 (如 `cbreak`)，使 CPU 暂停运行并跳转到调试器处理程序。

2. 基于 RISC-V 的 XV6 代码：描述特权模式 (Machine / Supervisor / User) 的转换过程。

开机阶段 (M → S)：CPU 启动后处于 Machine Mode。在 `start.c` 中，系统会配置 `mstatus` 寄存器将前一个模式设为 Supervisor，并设置 `mepc` 为 main 函数地址，随后执行 `mret` 指令，强制进入 Supervisor Mode 执行内核代码。

启动进程 (S → U)：内核完成初始化后，通过配置 `sstatus` (SPP 位设为 User) 和 `sepc` (指向用户程序入口)，执行 `sret` 指令降级进入 User Mode。

陷阱发生 (U → S)：当用户态发生系统调用或中断，硬件自动提升特权级至 Supervisor Mode，跳转到 `stvec` 指向的 trampoline 代码执行。

3. 为什么 `entry.S` 最后要写一个死循环 `spin`？如果没有这个循环会出现什么情况？

原因：`entry.S` 是多核 CPU 启动后的汇编入口。写 `spin` 循环是为了在某个 CPU 核心完成基础初始化 (如设置栈指针) 但尚未进入 `main` 函数，或是在后续运行中发生严重错误无法继续时，让该核心处于一个“原地踏步”的受控状态。

后果：如果没有这个循环，CPU 核心在执行完最后一条合法指令后，会继续从内存的下一个物理地址读取数据当作指令执行。由于该区域通常是随机数据，会导致 CPU “跑飞”，触发非法指令异常或产生不可预测的硬件行为，使整个系统崩溃。

4. 哪个函数处理了从用户空间发起的异常和系统调用？ (如 `usertrap`)

在 XV6 中，该函数是 `usertrap()` (位于 `trap.c`)。当发生陷阱时，汇编代码 `trampoline.S` 会先保存现场，随后跳转到此 `C` 函数。它负责识别 `scause` 的值，判断是系统调用 (调用 `syscall()`) 还是硬件中断 (调用 `devintr()`)，并进行相应处理。

5. 在处理陷入 (Trap) 时，使用了一系列控制寄存器，请说明 `scause`、`sepc`、`sstatus`、`stvec` 的作用？

`scause` (Supervisor Cause)：记录原因。保存导致陷阱发生的具体编号 (如 8 代表用户态环境调用 `ecall`)。

`sepc` (Supervisor Exception Program Counter)：记

录断点。保存发生陷阱时那条指令的地址，以便后续通过 `sret` 返回继续执行。

`sstatus` (Supervisor Status)：记录状态。保存当前的特权模式、中断使能状态等关键控制位。

`stvec` (Supervisor Trap Vector Base Address Register)：存储入口。保存内核陷阱处理程序的起始地址，硬件发生陷阱时会自动跳转到此处。

6. 系统调用结束以后，代码怎么准备跳回用户空间？ (如 `usertrapret`)

内核调用 `usertrapret()` 函数进行准备：重置寄存器：将 `stvec` 改回指向用户态陷阱入口。

填充 `Trapframe`：把内核栈指针、`usertrap` 地址等存入该进程的 `trapframe` 页，以便下次进入内核时使用。

设置状态：修改 `sstatus` 以便执行 `sret` 时能切换回用户模式，并将 `sepc` 设为之前保存的用户程序地址。

最后跳转：跳转到 `trampoline` 页面的汇编代码 `userret`，切换页表回到用户空间，恢复寄存器并执行 `sret`。

7. 不同 ISA (指令集架构，如 x86、RISC-V) 上怎样找到系统调用入口程序？

RISC-V：通过 `stvec` 寄存器。硬件发生 `ecall` 时直接跳转到该寄存器预设的固定地址。

x86 (早期)：通过 `IDT` (中断描述符表)。使用 `int 0x80` 指令根据索引号从表中查找入口。

x86 (现代)：通过 `MSR` (Model Specific Registers)。使用 `syscall` 指令直接从特定的机器状态寄存器中读取内核入口地址。

ARM：通过异常向量表 (Exception Vector Table)。发生 `svc` 指令时跳转到向量表中偏移量固定的对应位置。

Part 3 进程线程模型

1. 请调研 Linux 中是怎样管理 `task_struct` 的？

基本概念：Linux 内核使用 `task_struct` 结构体 (进程描述符) 来保存进程的所有信息 (如 PID、状态、优先级、地址空间、文件描述符等)。

内存分配：早期版本中，`task_struct` 存放在内核核的末尾；现代 Linux 使用 `slab` 分配器动态分配 `task_struct` 空间，并通过 `thread_info` 结构进行小部分底层信息的维护。

组织方式：任务列表 (Task List)：所有进程的 `task_struct` 被链接在一个全双工循环链表中。

哈希表：为了快速定位，内核维护了一个 PID 哈希表，可以通过 PID 直接找到对应的进程结构体。

树状关系：通过结构体内的指针维护父子 (parent/children) 和兄弟 (sibling) 进程关系。

2. 请调研协程、线程、C10K、I/O 编程模型的相关概念、应用场景。

线程 (Thread)：概念：进程内的执行流，共享进程的地址空间和资源，由操作系统调度。

应用场景：适用于多核 CPU 上的并行计算，或需要共享大量内存数据的任务。

协程 (Coroutine)：概念：轻量级、用户态的线程，切换由程序员

或运行时环境控制，无需内核参与，开销极低。

应用场景：高并发 I/O 密集型服务 (如 Go 语言的 `Goroutine`、Python 的 `asyncio`)。C10K 问题：

概念：指一台服务器同时处理一万个 (10,000) 并发连接的问题，核心瓶颈在于传统的“一连接一线程”模型导致的内存溢出和上下文切换开销。

I/O 编程模型：阻塞/非阻塞 I/O：调用程序是否挂起等待。

I/O 多路复用 (`select/poll/epoll`)：一个线程同时监听多个连接的状态，是解决 C10K 的主流技术。

异步 I/O (AIO)：操作系统完成数据读写后通知应用，效率最高但编程难度大。

Part 3 进程线程调度

1. FCFS、SJF、SRTN 和 RR 中，哪一个 (些) 是抢占式的？

抢占式 (Preemptive)：SRTN (最短剩余时间优先)：它是 SJF 的抢占版本。调度器会持续监控。如果新进入就绪队列的进程所需的运行

时间比当前正在运行进程的“剩余时间”还要短，系统会立即强行剥夺当前进程的 CPU，转交给新进程。

RR (时间片轮转)：这是一种基于时钟中断的强行抢占。系统为每个进程分配固定的时间片。一旦时间到，硬件中断会触发调度程序，将当前进程挂起并送回就绪队列末尾。

非抢占式 (Non-preemptive)：FCFS (先来先服务)：最简单的策略，进程一旦获得 CPU 就会运行到结束或主动阻塞，不可被中途夺权。

SJF (短作业优先)：在调度点选择执行时间最短的进程，一旦开始执行，即使后续有更短的任务进入，也不会被打断。

2. 某系统进行检测后发现，在阻塞 I/O 之前，平均每个进程运行时间为 T。一次进程切换需要的时间为 S，这里 S 实际上就是开销。对于采用时间片长度为 Q 的轮转调度，请给出以下各种情况的 CPU 利用率

的计算公式：

3. Linux 中 `need_resched` 的作用是什么？

`need_resched` 是 Linux 内核实现异步抢占的核心标志位 (Flag)。

作用：它充当了一个“预约通知”的角色。因为内核在执行关键代码时不能被随时中断，当调度器 (如在时钟中断里) 发现当前进程该让位了，它并不立即切换，而是先将该进程的 `need_resched` 置为 1。

功能：它确保了抢占只发生在安全的“调度点” (如从内核态返回用户态之前)，既保证了实时性，又保护了内核数据的一致性。

4. Linux 中 `need_resched` 在哪里？什么时候会设置？请举一个例子。

位置：通常存放在进程描述符 `task_struct` 关联的底层 `thread_info` 结构的 `flags` 字段中。

设置时机：时钟中断：周期性检查发现当前进程份额用尽。

唤醒逻辑：当一个更高优先级的进程由于 I/O 系统从睡眠中醒来。

调度实现：进程主动改变优先级或通过 `sched_yield()` 放弃 CPU。

例子：用户正在运行一个后台计算程序 A。此时用户移动了鼠标，触发了硬件中断。在处理鼠标中断的最后阶段，内核唤醒了前台的窗口管理程序 B。内核发现 B 的优先级更高，于是给程序 A 打上 `need_resched` 标记。中断处理一结束，内核检测到该标记，立即执行 `schedule()` 切换到 B。

5. 请指出 Windows 的调度算法与多级反馈队列 (MLFQ) 算法的 2 个相似点和 2 个不同点。

答案：

核心思路是计算一个周期内“有效工作时间 / 总时间”。

• (a) $Q = \infty$ ：利用率 $= \frac{T}{T+S}$ ，因为时间片无穷大，进程一定能跑完 T 时间直到 I/O 阻塞。

• (b) $Q > T$ ：利用率 $= \frac{T}{T+S}$ 。同上，因为进程在时间片用完之前就会因 I/O 而主动交出 CPU。

• (c) $S < Q < T$ ：利用率 $= \frac{T}{T+S \cdot \lceil \frac{T}{Q} \rceil}$ 。当 $Q < T$ 时，进程在完成 T 的任务中会被强行中断 $\lceil T/Q \rceil - 1$ 次，加上最后的 I/O 阻塞时间，总切换次数为 $\lceil T/Q \rceil$ 。

• (d) $Q = S$ ：利用率 $= 50\%$ 。此时 $\frac{T}{T+S \cdot \lceil \frac{T}{S} \rceil} = \frac{T}{2T} = 0.5$ 。一半时间在干活，一半时间在切换。

• (e) Q 趋近于 0：利用率 $= 0$ 。此时系统陷入“处理器抖动”，CPU 时间几乎全部浪费在上下文切换的开销 S 上。

相似点：

优先级队列：两者都将进程分为多个等级，且高优先级队列拥有绝对的执行优先权。

反馈动态调整：都根据进程的历史行为 (是计算密集型还是 I/O 密集) 来动态调整其所在的优先级层级。

不同点：

抢占时机：Windows 调度器支持即时抢占，只要高优先级线程就绪，低优先级线程会被立即中断；而传统 MLFQ 更多依赖时间片耗尽。

提振机制：Windows 拥有针对“前台窗口”和“用户交互”的特殊提振逻辑，而 MLFQ 通常只是一套通用的基于运行时间的数学惩罚机制。

6. Windows 中，哪类线程的优先级会提升？为什么？

提升类别：刚完成 I/O 等待的线程、刚被唤醒的互斥锁等待线程、处于前台活动窗口的线程、以及由于长期被忽略而产生饥饿的线程。

原因：响应性优化：I/O 线程和前台线程直接关系到用户的操作感受 (如输入反应、窗口刷新)。

吞吐量优化：让 I/O 线程尽快启动下一次 I/O 请求，能让硬件设备更忙碌，从而提高系统整体利用率。

安全性：防止由于优先级过低导致进程持有的锁无法释放，进而引发系统范围的死锁或阻塞。

7. 线程优先级在什么时机提升？提升后系统还有哪些相应设置？为什么？

提升时机：通常发生在内核函数 (如 `IoCompleteRequest`) 处理完事件并准备将线程设为就绪状态时。

相应设置：优先级衰减 (Decay) 机制。提升后的线程在每用完一个时间片 (量程) 后，其优先级会自动下降 1 级。

原因：这是为了防止“短期贡献者”变成“长期霸权者”。这种衰减确保了线程只会因为其最近的 I/O 行为获得暂时的奖励，最终还是会回归到设定的基本优先级，维持系统长期的公平性。

8. Windows 调度是怎样解决饥饿问题的？

Windows 依靠一个名为“平衡集管理器”(Balance Set Manager) 的后台线程。工作方式:它大约每隔 1 秒扫描一次系统的就绪队列。判断标准:寻找那些已经进入就绪状态、但由于优先级太低已经超过约 4 秒钟没能获得 CPU 的线程。

解决方案:管理器会将这些饥饿线程的优先级临时提升到 15 (动态优先级的最高级), 并为其分配两倍的时间片, 确保其能迅速处理完积压任务并释放资源。

9. 在多核处理器中, 为什么要考虑缓存亲和性和负载均衡等问题? 两者考虑的角度有什么不同?

原因: 在多核系统中, 数据被加载到某一个核心的 L1/L2 缓存中. 如果进程频繁切换到其他核心运行, 将面临巨大的“冷缓存”缺失开销。不同角度: 缓存亲和性 (Cache Affinity): 从执行效率出发。希望进程“从一而终”, 停留在原核心以利用热数据。负载均衡 (Load Balancing): 从吞吐量出发。希望所有核心“均贫富”, 避免一个核心累死, 另一个核心闲死。

权衡: 调度器必须在两者之间寻找最优解, 通常优先保证亲和性, 仅在负载严重倾斜时才进行迁移。

10. 调研: 火星探测器曾发生的优先级反转问题是怎样发生的?

低优先级 A 持有了关键资源的锁 (如文件锁)。高优先级 C 启动并请求该锁, 因此进入等待状态, 等待 A 释放。中优先级 B 此时启动。由于 B 的优先级高于 A, 且 B 不需要锁, 它抢占了 A 的 CPU。结果: 由于 B 一直在跑, A 没机会运行并释放锁, 导致最高优先级的 C 只能一直等下去。结局: 在火星探路者号上, 这导致高优先级任务超时, 触发了看门狗重置, 系统反复重启。

11. Linux CFS 调度: 新创建进程的 vruntime 的初值是不是 0?

不是。如果设为 0, 新进程会由于其虚拟运行时间 (vruntime) 极小, 被调度器认为极度“受委屈”, 而从长时间霸占 CPU。新进程的 vruntime 通常被设置为当前 CPU 运行队列中的 min_vruntime, 使其站在和大家相似的起跑线上。

12.Linux CFS 调度: 休眠进程的 vruntime 一直保持不变吗?

不是。休眠进程如果不调整 vruntime, 当它睡了很久醒来后, 其 vruntime 会远低于那些一直在运行的进程, 产生毁灭性的门槛占优势。因此在唤醒时, 内核会给它进行补偿, 将其 vruntime 更新到接近当前 min_vruntime 的水平。

13. Linux CFS 调度: 休眠进程在唤醒时会立刻抢占 CPU 吗?

不一定。CFS 遵循“最小粒度”原则。只有当被唤醒进程的 vruntime 确实小于当前运行进程, 并且两者的差值大于一个设定的门槛 (调度粒度) 时, 才会发生抢占。这能有效减少无意义的上下文切换, 保护缓存热度。

14. Linux CFS 调度: 进程从一个 CPU 迁移到

另一个 CPU 上的时候 vruntime 会不会变?

会变。因为不同 CPU 核心的负载情况不同, 它们的 min_vruntime 步调不一致。进程迁移时必须经历一个转换: 先减去原核心的 min_vruntime 转换为相对偏移量, 到达新核心后再加该核心的 min_vruntime。这确保了进程在新的环境下依然能获得公平的分配比例。

15. 内存管理概述

1. 何时将指令、数据绑定到内存地址? 有什么前提条件吗? (编译、链接、加载、运行)

编译/链接时 (Compile/Link time): 时机: 在程序编写完成后进行编译或链接时直接生成绝对地址。前提条件: 必须预先知道程序将驻留在内存的哪个特定位置。如果起始位置改变, 则必须重新编译。加载时 (Load time): 时机: 在程序从磁盘装入内存的过程中。前提条件: 编译器必须生成“可重定位代码”。如果装入位置改变, 只需重新加载并根据基址进行偏移。运行时 (Execution time): 时机: 程序在运行过程中才进行地址转换。前提条件: 必须有特殊的硬件支持 (如基址寄存器、界限寄存器以及 MMU 内存管理单元)。这是现代操作系统实现地址空间隔离和虚拟内存的基础。

期中考试真题整理:

2021 期中

一、环境和运行机制、系统调用 (30 分)

1. 请列举五条需要在保护模式下运行的指令。“访管指令”需要在保护模式下执行吗? 为什么? (10 分)

【参考答案】五条保护模式 (特权) 指令示例: lgdt (加载全局描述符表寄存器) lidt (加载中断描述符表寄存器) cli / sti (关中断/开中断) hlt (处理器停机) out / in (对 I/O 端口的输出输入, 通常在严格保护模式下受限) 注: 清内存、修改页表基址寄存器 (如修改 CR3) 也属于此类指令。“访管指令”是否需要在保护模式下执行? 不需要。

访管指令 (如 int n 或 syscall) 的作用是让程序从用户态切换到核心态, 请求操作系统提供服务。如果它只能在核心态 (保护模式/特权模式) 下运行, 那么运行在用户态的应用程序将永远无法调用系统服务, 这违背了该指令的设计初衷。

2. 描述从接收中断/异常到中断处理程序运行结束, 处理中断/异常的过程, 并指出哪些是硬件实现的? 哪些是操作系统实现的? 中断处理程序执行结束后, 需要执行哪一个模块 (哪一过程)? 操作系统初始化和中断有什么关系? (10 分)

【参考答案】处理过程及实现主体: 硬件实现 (中断隐指令阶段): 识别中断源, 获取中断号。

保护现场: 自动压入 PSW (标志寄存器)、PC (断点) 等关键寄存器到系统栈。根据中断向量表 (IVT/IDT) 找到对应处理程序的入口地址并跳转。切换 CPU 状态到核心态。操作系统实现 (中断处理程序阶段): 保护剩余的通用寄存器环境 (进一步保护环境)。执行具体的处理逻辑 (如驱动读写、异常修复)。恢复现场 (恢复通用寄存器)。执行中断返回指令 (如 iret)。

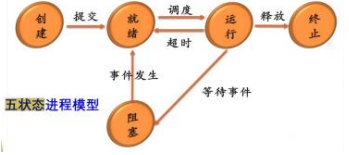
执行结束后的后续: 中断处理程序执行结束后, 通常会进入进程调度模块 (调度程序)。系统会决定是返回被中断的进程继续执行, 还是切换到更高优先级的进程。

操作系统初始化与中断的关系: 操作系统初始化的重要任务之一就是建立中断向量表/描述符表 (IDT)。在初始化完成前, 通常是关中断状态。只有当中断处理机制配置完成后, 系统才能通过中断实现并发、响应外设请求及系统调用。

3. 为了实现系统调用机制, 系统要做哪些工作? (10 分)

【参考答案】为了实现系统调用, 系统需要构建以下核心机制: 1 编写系统调用处理程序: 操作系统内部需实现各项功能的内核函数。2 设置系统调用入口 (中断向量): 指定一个特殊的中断号 (如 Linux 的 0x80) 作为系统调用的统一入口, 并将其注册到中断向量表中。3 建立系统调用号与处理程序的映射表: 内核维护一张表 (Syscall Table), 根据用户传入的编号找到具体的内核函数。4 参数传递机制: 规定用户程序如何将参数传给内核 (通常通过寄存器、栈或内存块)。5 陷入 (Trap) 机制: 硬件必须支持从用户态安全跳转到核心态, 并在跳转时自动完成权限提升和环境保存。二、进程线程模型 (20 分)

1. 请画出五状态进程模型, 包括状态的名称以及转移条件。(10 分)



五状态进程模型

【参考答案】

五状态进程模型包括: 新建态、就绪态、运行态、阻塞态 (等待态)、终止态。状态名称及转移条件: 新建态 (New) -> 就绪态 (Ready): 操作系统完成进程创建工作, 分配了除 CPU 以外的所有必要资源, 并将其加入就绪队列。就绪态 (Ready) -> 运行态 (Running): 进程调度程序选中该进程, 分配 CPU 时间片, 进程开始执行。运行态 (Running) -> 就绪态 (Ready): 运行进程的时间片用完, 或者由于出现了更高优先级

的进程而被抢占。运行态 (Running) -> 阻塞态 (Blocked): 进程请求某项资源或等待某个事件 (如 I/O 请求、申请信号量、等待子进程结束)。阻塞态 (Blocked) -> 就绪态 (Ready): 进程等待的事件已发生 (如 I/O 操作完成、信号量可用), 进程进入就绪队列等待调度。运行态 (Running) -> 终止态 (Terminated): 进程执行完毕或遇到不可恢复的错误, 被系统撤销并回收资源。

2. 线程和进程有什么区别? 线程有什么属性? 为什么线程要有自己的栈? 线程的实现方式有哪几种? 典型的操作系统是通过什么方式实现线程的? (10 分)

【参考答案】线程和进程的区别: 调度单位: 线程是独立调度的基本单位, 进程是拥有资源的基本单位。并发性: 进程之间可以并发执行; 同一进程内的多个线程也可以并发执行, 且切换开销远小于进程。资源拥有: 进程拥有独立的内存空间和系统资源; 线程基本不拥有系统资源, 但可以访问所属进程的资源。系统开销: 线程的创建、销毁和切换比进程快得多, 因为不涉及地址空间的重建。

线程的属性: 具有唯一的线程 ID。拥有独立的寄存器状态、程序计数器 (PC) 和栈。共享所属进程的地址空间、代码段、数据段和打开的文件等资源。

为什么线程要有自己的栈?

线程是执行的载体。为了支持独立的函数调用关系、保存局部变量以及函数返回地址, 每个线程必须拥有独立的栈, 以防止不同线程在执行过程中由于调用栈交织而导致数据混乱。

线程的实现方式:

用户级线程 (User-Level Threads, ULT): 由用户空间的线程库管理, 内核感知不到线程的存在。

内核级线程 (Kernel-Level Threads, KLT): 由操作系统内核直接管理和调度。组合/混合方式: 结合了 ULT 和 KLT 的优点 (如多对多模型)。

典型操作系统的实现方式:

现代典型操作系统 (如 Windows、Linux、macOS) 大多通过内核级线程 (KLT) 的方式来实现。在这种模式下, 内核可以直接调度单个线程, 利用多核 CPU 的并行能力。

三、进程调度 (25 分)

1. 现在有三个进程, A、B 和 C。A 和 B 的运行时间是 1000ms (全部在 CPU 上进行计算), C 进程是一个无限循环, 在每个循环中, 会先使用处理器 1ms, 然后再花 10ms 使用 I/O。请写出在以下几种调度算法下进程的调度状态, 并对每个调度算法对本任务的调度进行分析和评价。调度算法分别为: 先来先服务算

法、时间片轮转算法 (时间片分别为 100ms 和 1ms)、SRTN (最短剩余时间优先算法)。(10 分)

【参考答案】(1) 先来先服务算法 (FCFS) 调度状态描述: * 如果 A、B 先到达, 它们会各自连续占用 CPU 1000ms。在此期间, 进程 C 只能在就绪队列中苦苦等待, 直到 A 和 B 全部运行结束 (共计 2000ms 后) 才轮到 C。如果 C 先到达, 它运行 1ms 后进入 I/O, 此时 A 或 B 会抢占 CPU 并一直运行到结束, C 即使 I/O 完成也必须等 A、B 运行完。分析与评价: 评价: 极差。这种现象被称为“护送效应” (Convoy Effect)。长进程 (CPU 密集型) 霸占 CPU, 导致短进程 (I/O 密集型) 的周转时间过长, 且 I/O 设备在大部分时间里处于空闲状态, 系统资源利用率低。

(2) 时间片轮转算法 (RR, q=100ms) 调度状态描述: * A 和 B 轮流执行, 每次各占 100ms。进程 C 轮到时, 仅执行 1ms 就因为发起 I/O 而主动放弃 CPU 并进入阻塞态。10ms 后 I/O 完成, C 回到就绪队列等待下一个周期。分析与评价: 评价: 一般。相比 FCFS 有了很大进步, C 不再需要等待数秒。但因为 A、B 时间片较长, C 每次从 I/O 返回后, 仍可能要等待 A 或 B 运行完剩下的几十毫秒, 响应延迟仍有优化空间。

(3) 时间片轮转算法 (RR, q=1ms) 调度状态描述: * CPU 在 A、B、C 之间极快地切换。进程 C 每次执行 1ms 正好用完时间片 (或在此时发起 I/O), 进入阻塞态, 10ms 后返回就绪队列。A 和 B 轮流进行极细颗粒度的推进。分析与评价: 评价: 对 C 极好, 但系统整体效率低。C 的响应时间达到了最优。但由于时间片太小 (仅 1ms), 系统会频繁触发上下文切换, **上下文切换的开销 (Overhead) ** 相对于实际计算时间占比过高, 会显著降低 CPU 的有效利用率。

(4) 最短剩余时间优先算法 (SRTN) 调度状态描述: 进程 C 每次的 CPU 需求仅为 1ms, 其“剩余时间”始终是所有进程中最小的。因此, 只要 C 从 I/O 阻塞中返回并进入就绪队列, 它会立即抢占正在运行的 A 或 B。C 运行 1ms 后再次进入 I/O, CPU 随即还给 A 或 B。

分析与评价: 评价: 最优。这种算法完美兼顾了各方需求: 它保证了 I/O 密集型进程 (C) 的最高优先级, 使得 I/O 设备和 CPU 都能保持高效率的并发工作, 同时也最大限度地减少了平均等待时间。

三、进程调度 (25 分)

2. 请设计一个基于多级反馈队列的调度算法, 描述你的算法并回答以下问题: (15 分)

【算法描述】: 设置 N 个就绪队列, Q1 优先级最高, 随序号增加优先级降低, 时间片随序号增加而翻倍 (如 Q1 = 10ms, Q2 = 20ms ...)。新进程进入 Q1, 若在时间片内未完成则降级至下一级队列。

【问题回答】:

(1) 你设计的算法是抢占的还是非抢占的? 为什么? 抢占式。 当高优先级队列 (如 Q1) 出现新进程或 I/O 返回的进程时, 应立即中断当前低优先级队列进程的执行。这是为了保证短进程和交互式进程的响应速度。

(2) 你设计的算法对 I/O 密集型和 CPU 密集型进程哪个更友好? 说明你的原因。对 I/O 密集型进程更友好。 因为 I/O 密集型进程 (如进程 C) 通常在极短的时间片内就会因 I/O 阻塞而主动放弃 CPU, 这使得它能长期停留在高优先级队列中; 而 CPU 密集型进程会因耗尽时间片而不断被降级。

(3) 你是如何处理进程等待 I/O 后再次被调度上 CPU 时要继续运行的时间片大小的? 通常有两种做法: 保持优先级: 进程回到阻塞前的那个队列, 并赋予该队列完整的起始时间片。提升优先级: 考虑到该进程可能属于交互型, 可将其提升回最高优先级队列 Q1, 以获得最快响应。

(4) 你设计的算法会导致“饥饿”问题吗? 为什么会有这个问题? 如果产生了“饥饿”的问题, 要用什么方法解决? 会。 若高优先级队列始终有新进程或 I/O 频繁返回的进程, 低优先级队列中的长进程 (CPU 密集型) 将永远得不到 CPU 资源。解决方法: 引入 ***提升优先级 (Priority Boosting)*** 机制。每隔一段固定的时间 (S), 将系统中所有进程强制移回最高优先级队列 Q1, 确保长进程也能定期获得执行机会。

2019 期中

一、运行环境和运行机制 (30 分)

1. CPU 状态是软件实现的还是硬件实现的? 为什么这么实现? (4 分)

【参考答案】实现方式: 硬件实现。 通常通过 CPU 内部的程序状态字寄存器 (PSW) 中的特权位 (Mode Bit) 来标识。

原因: 安全性: CPU 状态控制着指令的执行权限 (如特权指令)。如果由软件实现, 恶意软件可以通过修改软件标识来获取最高权限, 硬件实现可以强制执行访问控制。效率: 硬件可以在指令译码阶段直接检查当前状态, 不增加软件调用的额外开销。

2. 举例说明中断/异常的区别。(4 分)

【参考答案】中断 (Interrupt, 外中断): 源于 CPU 外部的信号。例子: 键盘敲击、时钟中断、I/O 操作完成信号。异常 (Exception, 内中断): 源于 CPU 内部执行指令时产生的错误或特殊请求。例子: 除以零、缺页异常、执行系统调用指令 (访管指令)。核心区别: 中断是异步的 (随时可能发生, 与

当前指令执行无关），异常是同步的（在特定指令执行时触发）。

4. 系统调用有什么作用？它和函数调用有什么区别？（4 分）

【参考答案】
作用：为应用程序提供访问内核资源的统一接口（如文件、进程、网络）。
保证系统的安全性和稳定性，防止用户程序直接操作敏感硬件。
区别：
运行状态：函数调用在同态进行（如都在用户态）；系统调用涉及用户态到核心态的转换。
控制转移：函数调用地址固定（编译时决定）；系统调用通过“陷入（Trap）”机制，通过中断向量表跳转。
返回方式：系统调用返回时可能触发进程调度。

5. 操作系统是如何实现系统调用的？（8 分）

【参考答案】
陷入指令：用户程序执行访管指令（如 int 0x80），产生异常。
硬件支持：CPU 自动切换到核心态，并根据中断向量表跳转到统一的系统调用入口地址。
内核处理：
查表：根据寄存器中的系统调用号（Syscall ID）查找系统调用表。
执行：调用对应的内核服务函数。
传参：通过寄存器或系统栈完成参数传递。
返回：执行特权指令切换回用户态，返回用户空间执行后续指令。

二、 进程线程模型（25 分）

1. （8 分）PCB 的作用是什么？PCB 里保存了哪些内容？PCB 和进程的调度/切换有什么关系？PCB 是如何保存进程地址空间的？

【参考答案】
PCB 的作用：PCB（进程控制块）是进程存在的唯一标志。它使一个在多条程序环境下不能独立运行的程序（含数据）成为一个能够独立运行的基本单位，一个能与其他进程并发执行的进程。

保存的内容：
进程标识符：PID(进程 ID)、UID(用户 ID)。处理机状态信息：通用寄存器、指令计数器（PC）、程序状态字（PSW）、栈指针。
进程调度信息：进程状态、优先级、调度算法相关参数、阻塞原因。
进程控制信息：程序和数据的地址、资源清单（如打开的文件）、进程间同步机制。

与调度/切换的关系：进程切换时，内核会将当前进程的 CPU 现场（上下文）保存到该进程的 PCB 中，并从下一个被调度进程的 PCB 中恢复现场。PCB 是上下文切换的物理载体。

如何保存地址空间：PCB 中通常包含一个指向页表（Page Table）基址的指针（在 x86 中对应 CR3 寄存器的值），通过这个指针，内核可以在切换进程时切换虚拟内存映射，从而实现地址空间的隔离与保存。

2. （4 分）进程是如何创建的？

【参考答案】
进程创建通常遵循以下步骤：
申请空白 PCB：为新进程分配唯一的进程标识符。
分配资源：为进程分配必要的内存空间（代码、数据、堆栈等）。
初始化 PCB：初始化标识信息、处理机状态信息（设置 PC 指向程序入口）和控制信息。
插入就绪队列：将 PCB 插入就绪队列，等待调度程序调度。
注：在类 Unix 系统中，通过 fork() 拷贝父进程创建子进程，再通过 exec() 加载新程序。

4. （4 分）用户线程和核心线程有什么区别？

【参考答案】
管理与调度主体：
用户级线程：线程管理的所有工作（创建、撤销、切换）都由用户空间的线程库完成，操作系统内核完全感知不到线程的存在，仍以进程为单位进行调度。
内核级线程：线程的管理由操作系统内核直接负责，内核维护线程的上下文信息，调度是以线程为基本单位进行的。
阻塞后的系统表现：
用户级线程：如果一个线程因 I/O 等原因进入阻塞，由于内核只看得到进程，会导致整个进程及其所有线程都被阻塞。
内核级线程：如果一个线程阻塞，内核可以调度同一进程中的其他线程继续运行，不会导致整个进程挂起。

对多核 CPU 的利用：
用户级线程：无论有多少个核心，一个进程的所有用户线程都只能被分配到同一个 CPU 核心上执行，无法实现真正的并行。
内核级线程：内核可以将同一个进程中的多个线程分派到不同的 CPU 核心上并行执行，充分利用多核优势。

执行效率与开销：
用户级线程：线程切换发生在用户态，不需要模式切换，因此切换开销极小。
内核级线程：线程切换需要从用户态陷入内核态，由内核完成上下文保存和恢复，系统开销相对较大。

2018 期中

一、 关于 CPU 状态（8 分）

1. 为什么要将指令集合划分为不同的特权级别？

答：为了保护系统安全和稳定运行。高特权级（内核态）执行关键操作，低特权级（用户态）执行普通应用程序，防止用户程序直接访问硬件资源或执行危险操作，避免系统崩溃。

2. 下列指令哪些指令只能在内核态（管态）下执行？

（算术运算、内存读写、读日期时间、启动 I/O、修改程序状态字、访管指令、陷入指令、允许/禁止中断、取数据指令）
答：启动 I/O、修改程序状态字、允许/禁止中断。

注意：访管指令和陷入指令是用户态执行的，用来进入内核态。

二、 关于中断、异常和系统调用（15 分）

1. 在实现系统调用时，需要硬件体系结构提供哪些支持？在操作系统中应完成哪些工作？以 IA32 为例描述执行流程。

硬件支持：特权级机制、中断门机制（IDT）、寄存器保存现场。
操作系统工作：维护系统调用表、实现内核处理函数。
流程：用户程序设置系统调用号（EAX）和参数 -> 执行 int 0x80 -> CPU 切换到内核态，压栈保存现场 -> 根据 IDT 找到入口 -> 根据 EAX 查表执行内核函数 -> 返回用户态并传回结果。

2. 系统调用与函数调用有什么区别？系统调用与 API 是什么关系？

答：区别：见 19 年期中第一大题 q4
关系：API 是接口规范，系统调用是内核服务。通常 API 封装了系统调用。

三、 关于进程线程模型（18 分）

2. 操作系统对用户级线程和内核级线程的支持有什么不同点？各列举一个实例。

第一问见 19 年第二大题 q4
用户级：实例：早期的 Solaris。
内核级：实例：Linux（NPTL）。

四、 关于进程调度（18 分）

1. 哪一种进程调度算法会导致优先级反转问题？请举例描述并说明如何解决。

答：抢占式优先级调度。
现象：低优先级 L 持有资源，高优先级 H 等待 L；此时中优先级 M 抢占 L 运行，导致 H 必须等待 M 运行完，发生了优先级倒置。
解决：优先级继承协议（L 临时继承 H 的优先级）或优先级天花板。

六、 综合题（17 分）

1. 叙述从命令行输入执行程序 A，到 A 执行完系统调用 read 并退出的全过程。
答：Shell fork 子进程 -> exec 加载 A -> A 运行 -> 调用 read -> 陷入内核 -> A 阻塞，切换到进程 Z -> 磁盘 I/O 完成产生中断 -> A 回到就绪态 -> A 获得 CPU 继续运行 -> 调用 exit 退出 -> 父进程回收资源。
2. 设计操作系统时要对“时间与空间”、“公平与性能”进行权衡，请各举一例。
答：时空权衡：页面大小。小页面减小内部碎片（省空间）但增加页表项和换页次数（费时间）。
公平性能权衡：调度算法。轮转调度（RR）保证公平，但频繁切换降低性能；优先级调度追求效率但可能导致不公平。

xv6 代码部分

PPT0

1、内核文件是如何被加载和“运行”的？如何确定它的第一行代码从哪开始？

运行机制：内核不像普通用户程序那样从 main() 被直接调用，而是由硬件或启动链固件（如 Bootloader / OpenSBI / RustSBI）将其镜

像加载到内存指定位置。固件在 M-mode（机器态）完成极早期初始化后，跳转到内核入口。

确定起点（入口点）：起点由链接脚本 kernel.ld 决定。通常使用 ENTRY(_entry) 声明。因此，CPU 第一次进入内核时，是从汇编符号 _entry 开始取指执行的。

典型控制流：kernel.ld (ENTRY(_entry)) -> _entry (entry.S) -> start (start.c) -> main (main.c)。

2、start 函数运行在什么模式？它是如何跳转到内核的 main() 的？

运行模式：start() 运行在 M-mode（机器态）。因为它需要配置只有 M-mode 才有限权修改的硬件资源（如设置中断委托 medeleg/mideleg 等）。

跳转机制（核心考点）：

通过配置 CSR 寄存器并使用 mret 指令完成模式降级和跳转：
将 main 函数的地址写入 mepc 寄存器：w_mepc(uint64_t main);
将 mstatus 寄存器的 MPP 位设置为 S（Supervisor）。

执行 mret 指令。此时 CPU 会特权降级到 S-mode（监督者态），并将 PC 设置为 mepc 的值，从而跳转到 main() 继续执行。

3、在多核环境下，各 CPU 核心（hart）的内核栈是如何排列和分配的？

分配原理：xv6 在 entry.S 的数据段预留了一块连续的 stack0（或称为 bootstack）大数组作为启动栈空间。

计算公式（重点）：利用当前的 CPU 的核号 hartid（从 0 开始）来计算各自的栈顶地址：

sp = stack0 + (hartid + 1) * STACK_SIZE
内存布局：结果就是各个核的栈在内存中是连续排列的（[CPU0 栈][CPU1 栈][CPU2 栈]...）。因为栈默认“向下增长”，所以每个核利用计算出的高地址（栈顶），往低地址方向安全压栈。

4、简述 xv6 多核启动流程（main.c 中的初始化过程）？

所有核心上电后几乎同时进入 _entry，设置好各自的独立栈后进入 main.c，并分工执行：

CPU 0 (hart0) 执行全局初始化：完成各种全局单例设施的初始化（如 consoleinit, kinit 物理页分配器, kvminit 内核页表, procinit 进程表等）。

调用 userinit() 创建系统的第一个用户进程（init 进程）。

设置全局同步标志：started = 1（通过这个标志告诉其他核可以启动了）。

其他 CPUs (hart 1..n) 等待并执行每核特有的初始化逻辑（如开启本核的分页 kvmnithart、设置本核的陷入向量与中断 trapinithart 或 plicnithart）。

终点：
所有核心（包括 CPU 0）最终都会调用 scheduler() 函数进入死循环，系统正式进入正常的多核进程调度运行态。

PPT2 启动与中断异常机制

1、xv6-riscv 有几个特权模式？启动阶段的特权模式是如何转换的？

xv6-riscv 涉及 3 个特权模式：

M-mode (Machine)：最高特权级，通常运行 Bootloader/固件（如 OpenSBI/RustSBI）。

S-mode (Supervisor)：内核态，xv6 kernel 运行在此模式。

U-mode (User)：用户态，普通用户进程运行在此模式。

启动阶段特权级转换过程（Boot）：

M-mode -> S-mode（固件切换到内核）

位置：start.c 中的 start() 函数（或通过固件完成）

过程：

设置 mstatus 寄存器的 MPP 位为 S（表示 previous mode 为 Supervisor）。

将 mepc 寄存器设置为 main 函数的地址。

执行 mret 指令。CPU 会将 PC 设置为 mepc，并根据 MPP 降级到 S-mode，进入 main.c 中的 main() 执行内核初始化。

S-mode -> U-mode（内核初次进入用户态）

位置：trap.c 的 usertrapret() 函数与 trampoline.S 的 userret。

过程：

清除 sstatus 的 SPP 位（设为 0），表示上一次特权级为 U-mode。

将 sepc 寄存器设置为用户程序的入口地址（tf->epc）。

跳转到 trampoline.S 中的 userret 汇编代码，最后执行 sret 指令，CPU 从 S-mode 切换到 U-mode。

2、xv6-riscv 进行系统调用时，系统调用号、参数及返回值分别通过哪些寄存器传递？

遵循 RISC-V 约定，相关处理位于 kernel/syscall.c 和 trampoline.S。

系统调用号：a7 寄存器（内核中通过 p->trapframe->a7 获取）。

参数传递：最多 6 个参数，依次使用 a0, a1, a2, a3, a4, a5 寄存器。

返回值：存入 a0 寄存器。（系统调用结束时，内核通过 p->trapframe->a0 = 返回值 写回，用户态读取 a0 获取结果）

3、xv6-riscv 在进行进程上下文切换/Trap 时，需要保存和恢复哪些寄存器？

需要严格区分进程调度切换 和 用户态陷入内核 (Trap) 两种情况：

内核级线程调度切换 (Context Switch)

位置：kernel/switch.S

保存内容：仅保存 Callee-saved（被调用者保存）寄存器。

具体寄存器：ra（返回地址）、sp（栈指针）、s0~s11（保存的寄存器，其中 s0 也是 fp）。

存放位置：进程的 struct context 结构体。

用户态陷入内核 (User Trap)

位置：kernel/trampoline.S (uservec 和 userret)

保存内容：几乎所有的通用寄存器 (GPR) + 用户 PC，因为必须保证能“原样”回到用户态继续执行。

具体寄存器：ra, sp, gp, tp, t0~t6, s0~s11, a0~a7，以及 sepc（发生 trap 时的用户 PC）。

存放位置：进程的 struct trapframe（陷入帧）。注：内核态 Trap(kernelvec.S) 类似，在当前内核栈上保存几乎全套 GPR。

4、处理 Trap 时以下控制寄存器（stvec, sepc, scause, sstatus）的作用是什么？

这些是 RISC-V 的核心 CSR（控制与状态寄存

器），处理逻辑主要在 trap.c:

stvec (Supervisor Trap Vector)

作用：存放 Trap 处理程序的入口基地址。

应用：内核态下设为 kernelvec；准备返回用户态时（usertrapret）设为 uservec（位于 trampoline 页）。

sepc (Supervisor Exception PC)

作用：记录 Trap 发生时的 PC 值（即异常返回地址）。

应用：系统调用时，sepc 指向 ecall 指令本身，内核需手动 tf->epc += 4 让其返回后执行下一条指令。

scause (Supervisor Cause)

作用：记录发生 Trap 的原因编码。

应用：最高位为 1 表示中断 (Interrupt)，为 0 表示异常/系统调用 (Exception)。例如：scause == 8 表示来自 U-mode 的系统调用 (ecall)。

sstatus (Supervisor Status)

作用：记录和控制处理器的当前运行状态。关键位：

SPP (Supervisor Previous Privilege)：记录 trap 前是 S 态(1) 还是 U 态(0)。usertrap() 通过检查 SPP==0 确保来源是用户态。

SIE (Supervisor Interrupt Enable)：当前是否允许 S-mode 中断。

SPIE (Supervisor Previous IE)：记录 Trap 发生前的中断使能状态，sret 时会恢复。

5、简述 devintr() 和 uartintr() 的内部流程，及 devintr 调用 uartintr 的判断条件？

相关代码位于 kernel/trap.c 和 kernel/uart.c。

1. devintr() 内部流程与判断条件 (Trap 分发)

流程：读取 scause 判断中断类型。

若为外部中断 (S-mode External Interrupt)：调用 plic_claim() 获取中断号 irq。根据 irq 分发给相应设备驱动处理，处理完调用 plic_complete() 结束，返回 1。

若为时钟中断 (S-mode Timer Interrupt)：调用时钟处理函数，返回 2。

否则返回 0（非设备中断）。

调用 uartintr() 的判断条件：

获取到的外部中断号 irq 等于 UART 设备对应的中断号。

// devintr() 中的关键判断代码
int irq = plic_claim();

if(irq == UART0_IRQ){
 uartintr(); // 条件满足，调用 UART 中断处理
}

2. uartintr() 内部流程 (UART 中断处理)

分为输入和输出两个半部处理：

输入处理 (RX)：通过 uartgetc() 循环读取接收寄存器中的字符（直到读到 c）。每读到一个字符 c，就调用 consoleintr(c) 丢给控制层处理（如回显、处理退格、唤醒等待读取的进程）。
输出处理 (TX)：获取 UART 锁，调用 uartstart()。检查发送缓冲区中是否有待发送的字符，如果有且硬件发送空闲，则将字符写入发送寄存器进行输出。

PPT3

1、Shell 是如何执行一条命令的？（典型执行链路）

Shell 运行命令的核心链路是：读入命令 -> 解

析 → fork() 出子进程 → 子进程 exec() 替换程序 → 父进程 wait() 回收子进程。

具体分为两种情况（参考 sh.c 的主循环）：

内建命令（如 cd）：必须在父进程（shell 本身）直接执行。因为 chdir 修改的是当前进程的工作目录，如果在子进程中执行，子进程退出后 shell 的工作目录并没有改变。

外部命令（如 ls, cat）：调用 fork() 创建子进程，在子进程中调用 runcmd(cmd)，进而触发 exec(); 父进程则调用 wait() 阻塞等待。

2、fork()、exec() 和 wait() 各自的具体功能是什么？（结合 xv6 源码）

fork() —— 复制进程

作用：复制出一个与父进程“几乎一模一样”的子进程（拷贝了地址空间内容、打开文件表、当前工作目录等）。

区别：父子进程唯一的区别是 fork() 的返回值。源码细节（proc.c）：xv6 在内核中通过修改子进程的陷入帧，强行让其返回值为 0：

```
np->trapframe->a0 = 0;
```

exec(path, argv) —— 替换映像

作用：不创建新进程（PID 不变），而是将当前进程的用户态内存（代码/数据/栈）替换为新程序的映像。

特征：成功执行后，不返回到原程序的代码处。只有 exec() 失败时，才会返回并打印错误退出（参考 sh.c 中 runcmd 的失败处理）。

wait(&status) —— 阻塞与回收

作用：父进程阻塞等待属于自己的子进程退出。源码细节（proc.c）：子进程调用 exit() 后，状态会变为 ZOMBIE（僵尸进程），保留退出码。只有当父进程调用 wait() 时，才会扫描进程表，获取其退出码并最终调用 freeproc() 彻底释放子进程的内核资源（防止资源泄漏）。

3、Unix/xv6 将进程创建设计为 fork() 和 exec() 分离，这种设计有什么好处？

这是《Three Easy Pieces》中强调的经典 Process API 设计哲学：

1) 解耦“创建进程”与“装载程序”：

fork() 只负责准备一个可运行的执行上下文，而 exec() 只负责将该上下文变成想运行的目标程序。

2) 赋予了极强的“组合能力”（支持重定向与管道）：

因为 fork() 和 exec() 是分开的，Shell 可以在这两者之间的“间隙”做大量的环境配置，如修改文件描述符（FD）。

I/O 重定向：fork() -> 子进程 close(1); open("out.txt") -> exec()。目标程序毫不知情，但输出已经被重定向。

管道 (PIPE)：两个 fork() 分别跑左右命令，子进程在 exec 之前先 dup/close 修改 FD，极其自然地实现了 stdout → pipe → stdin（参考 sh.c 的 case PIPE。）。

3) 保持父进程状态的持续性：

Shell 作为父进程负责展示提示符、读取输入和管理 job。每条外部命令都在子进程里 exec，这样不会把 Shell 进程本身给替换掉，Shell 可以一直存活。

PPT4 进程线程调度

1、xv6 调度的核心数据结构有哪些？分别保存了什么关键信息？

主要在 kernel/proc.h 中定义了三个核心结构：

struct context (内核线程上下文)：专给 switch() 函数使用（仅保存 Callee-saved (被调用者保存) 寄存器：ra, sp, s0~s11）。

struct cpu (CPU 调度器上下文)：

proc：当前 CPU 正在运行的进程指针。

context：scheduler() 自己的上下文（进程切回调度器时需恢复到这里）。

noff/intena：用于 push_off() / pop_off() 记录中断嵌套深度与原始中断使能状态。

struct proc (进程控制块 PCB)：

调度相关：state（UNUSED/ SLEEPING/ RUNNABLE/ RUNNING / ZOMBIE）、context、kstack。

陷入/页表：trapframe、pagetable（用户页表）。（注：在某些 xv6-k210 变种中还有 kpagetable，即每进程有独立的内核页表）。

2、switch(old, new) 进行内核上下文切换时，到底保存/恢复了哪些寄存器？为什么？

操作（kernel/switch.S）：将当前寄存器保存到 *old，并从 *new 恢复寄存器。

保存的寄存器：ra, sp, s0~s11。

为什么不保存 a0~a7 或 t0~t6？

因为它们是 Caller-saved (调用者保存) 寄存器，按照 C 语言函数调用约定，在调用 switch() 之前编译器已经生成了保存它们的代码。switch 的目的是在内核 C 控制流之间切换，只需保证“能回到切换前的内核代码继续跑”即可。

3、scheduler() 主循环是如何工作的？

每个 CPU 启动后都会运行一个 scheduler() 函数，且永不返回（kernel/proc.c）。

其核心逻辑如下：

开中断：调用 intr_on() 允许设备中断，防止系统“死等”。

扫描进程表：寻找状态为 RUNNABLE 的进程。

执行切换：

获取该进程的锁：acquire(p->lock)。

修改状态与绑定：p->state = RUNNING; c->proc = p;。

(k210 变种特有：切换 satp 到 p->kpagetable)。

核心切换：switch(&c->context, &p->context)，跳转到进程的内核上下文执行。（进程将来通过 sched() 切回后，从这里继续向下执行），释放锁 release(p->lock)。

4、从进程切回调度器的 sched() 函数有哪些严格的调用前提？（△极易考）

sched() 的作用是保存当前 CPU 的 intena，通过 switch(&p->context, &mycpu()->context) 切回调度器。

调用它必须严格满足以下前提条件（否则会 panic）：

必须持有且仅持有当前进程的锁：p->lock。

当前进程状态必须非 RUNNING：必须已提前被修改为 RUNNABLE、SLEEPING 或 ZOMBIE。

必须处于关中断状态：intr_get() == 0，避免在切换途中发生中断破坏状态。

5、让出 CPU (yield) 以及退出/回收 (exit/wait) 在调度上的体现是什么？

yield()（主动让出时间片）：

获取 p->lock -> 设状态为 RUNNABLE -> 调用 sched() 切走 -> 切回来后释放 p->lock。

exit(status)（进程退出）：

关闭资源、将子进程过继给 init 进程 (reparent)

-> 唤醒父进程 -> 设状态为 ZOMBIE -> 调用 sched() 切走（此后该进程永远不会返回）。

wait(status)（回收僵尸进程）：

父进程持有 p->lock 扫描子进程：

找到 ZOMBIE 子进程：彻底回收 (freeproc) 并返回其 pid。

没找到且还有存活的子进程：调用 sleep(p, &p->lock) 等待子进程 exit() 唤醒自己。

6、sleep() 和 wakeup() 是如何配合的？如何避免“丢失唤醒 (Lost Wakeup)”？

sleep(chan, lk) 的核心目标是：保证“释放业务锁 lk”与“进入睡眠”这两个动作的原子性。

实现：先获取 p->lock，然后再释放业务锁 lk（顺序极度重要！这就保证了此时哪怕发生中断，wakeup 也拿不到 p->lock）。接着设置 p->chan = chan，p->state = SLEEPING，最后调用 sched() 切走。醒来后再清空 chan 并重新获取 lk。

wakeup(chan) 的逻辑：

遍历进程表，尝试获取每个进程的 p->lock。

如果发现 p->state == SLEEPING 且 p->chan == chan，就将其状态改为 RUNNABLE。

第二章 操作系统运行环境与运行机制

零、 核心设计哲学

机制与策略分离原则 (Separation of Mechanism and Policy)：

机制（硬件/OS 底层提供）：如中断机制、特权级切换。

策略（OS 高层决定）：如调度算法选哪个进程运行。

ECF (Exceptional Control Flow，异常控制流)：是理解应用程序与 OS 交互、并发、软件异常的核心钥匙。

一、 处理器状态与特权级

硬件机制将操作系统与用户程序隔离，其核心是处理器特权级别。

通过执行特权状态字（PSW）中的特定位来区分。

1. 体系结构对比

x86 架构：支持 4 个特权环（R0 ~ R3）。从 R0（最高权限）到 R3（最低权限）。

Linux 实际使用：通常只使用 R0（内核态 / Supervisor mode）和 R3（用户态 / User mode）。

RISC-V 架构：三种特权模式。

‘M’ (Machine, 机器模式) 级别 3

‘S’ (Supervisor, 监管模式) 级别 1

‘U’ (User, 用户模式) 级别 0

2. 状态转换

用户态 → 内核态：唯一途径是中断/异常/陷入机制 (Trap)。

内核态 → 用户态：通过设置程序状态字 (PSW——RISC-V 的状态字) 或专用返回指令实现。

陷入指令 (Trap Instruction)：也称访管指令，是用户请求 OS 服务的接口（如 ‘int’, ‘trap’, ‘syscall’, ‘ecall’ ——这两条是用来系统调用的，

前者为 x86，后者为 RISCv/xv6）。

二、 中断与异常机制 (Interrupts & Exceptions)

操作系统是由中断/异常/事件驱动的：处理中断请求；捕获用户程序提出的服务请求；防止用户程序的破坏性活动。

1. 事件的分类 (绝对高频考点)

类别	原因	异步/同步	返回行为
中断 Interrupt	来自 I/O 设备、其他硬件部件	异步	总是返回到下一条指令
陷入 Trap	有意识安排的	同步	返回到下一条指令
故障 Fault	可恢复的错误	同步	返回到当前指令
终止 Abort	不可恢复的错误	同步	不会返回

2. 软硬件的明确分工

硬件负责 (中断响应)：

x86:

在每条指令执行周期最后扫描中断寄存器，检查是否有中断信号。

发现并接收中断。

切换处理器特权态（用户态 -> 内核态）。

保存现场 (压栈): 保存 PC、PSW (EFLAGS) 这两个概念只在 IA32 的实模式下使用，不支持权限改变。

等核心寄存器到系统栈。如果是异常，还可能压入错误码 (Error Code)。

硬件通过查询中断向量（保存中断处理程序入口地址+程序运行所需的处理机状态字）表 / 中断描述符表 (IDT)，将 PC 指向中断处理程序入口。

RISCv:

硬件把旧 PC 存入 ‘sepc’，然后把 ‘stvec’ 里的地址直接丢给真实的 PC。

软件负责 (中断处理)：

进一步保存通用寄存器现场信息 (如 ‘SAVE_ALL’ 宏)。

在 RISCv (xv6) 中：若为用户态触发中断，则系统会执行 ‘trampoline.S’，将其余的寄存器值存在 ‘proc’ 结构体的 ‘trapframe’ 中。若为内核态时再次引发中断，此时 ‘stvec’ 的内容已被 ‘usertrap()’ 改为了 ‘kernelvec.S’，因此会在内核栈上保存其余寄存器。

分析原因，执行对应设备的逻辑或错误恢复。

恢复现场（除了 PSW 和 PC 以外的寄存器）。

决定是否进入调度，如果否，直接执行 ‘iret’ 指令恢复运行。

硬件负责（最终恢复）：

恢复 PSW 和 PC，根据 PSW，将 cpu 调整回用户态。

根据 PC 开始下一个指令周期。

3. IA-32 (x86) 中断实现细节 (考点)

x86 处理器的基本概念：

中断：由硬件引发、异步、分为可中断不可中断。

异常：由指令执行引发，对于某些异常，CPU 会在执行异常处理程序之前产生硬件出错码，并压入内核态堆栈。

系统调用：异常的一种，用户态到系统态唯一入口

实模式：由 PIC 转换中断信号为中断向量，使用中断向量表。

\$入口地址 = 段地址 << 4 + 偏移地址\$，

不支持状态切换（因此中断处理与一般的过程调用相似）。

保护模式：使用中断描述符表 (IDT)，通过 IDTR 寄存器定位。包含多种门 (Gate)：

struct. 中断发生时，将取代当前进程的运行进程 TSS 选择符存放其中。

中断门 (Interrupt Gate)：进入后系统会自动禁止中断（清空 IF 位）。

陷阱门 (Trap Gate)：进入后系统不会自动关中断。*考点：这是中断门和陷阱门的核心区别！

系统调用 (int 0x80) 使用的是陷阱门！*

调用门。

中断向量/中断向量表：

“中断向量号”是一个数值，用以索引“中断向量”是一个内存单元，为中断向量表的条目，存放中断处理程序入口地址和程序运行所需的处理机状态字。

这两个概念只在 IA32 的实模式下使用，不支持权限改变。

中断描述符/中断描述符表：

确定中断或异常的向量 i（中断向量号）

IDTR 寄存器存放 IDT 表，得到中断描述符

GDTR 获得 GDT 地址，结合中断描述符的段选择符和段内偏移量，找到入口程序

三、 系统调用 (System Call)

1. 基本概念

系统调用与 API 的区别 (经典问题)：

系统调用是 OS 提供的唯一内核接口，执行会触发态的切换（陷入内核）。

C 库函数 / API 是在用户态的函数库，它们内部可能会封装并触发系统调用（如 ‘printf’ 内部触发 ‘write’），也可能完全不涉及系统调用（如 ‘strlen’、‘abs’）。

机制与策略：提供系统调用的陷入门是机制，而各个具体的系统调用函数（如 ‘sys_write’）的实现逻辑是策略。

2. x86 系统调用完整执行流程 (‘int 0x80’)

- 参数传递：用户程序将参数放入通用寄存器。‘eax’ 存系统调用号，‘ebx, ecx, edx, esi, edi’ 存参数 (最多 6 个)，或者在内存中专门开辟堆栈区用于传递。
- 触发陷入：执行 ‘int 0x80’。
- 硬件响应：查 IDT 表的 128 (0x80) 号门 (陷阱门，DPL=3 允许用户态调用)。特权级切换，切换栈（用户栈换为内核栈），硬件自动将用户 ‘SS’、‘ESP’、‘EFLAGS’、‘CS’、‘EIP’ 压入内核栈。
- 软件总入口：跳转到 ‘system_call’。执行 ‘SAVE_ALL’（压入其他寄存器如 ds, es, eax 等）。
- 查表分派：检查 ‘eax’ 中的调用号是否合法，查 ‘sys_call_table’，跳转到对应的内核处理函

数（如 ‘sys_write’）。

6）返回过程：内核函数执行完毕，结果存入 ‘eax’。进入 ‘ret_from_sys_call’，检查是否需要重新调度（‘need_resched’ 或处理信号）。

7）恢复用户态：执行 ‘RESTORE_ALL’ 和中断返回指令，弹栈恢复寄存器，回到用户空间。

3. RISCv 上执行系统调用

约定:遵循标准的 RISC-V ABI(Application

Binary

Interface)。

系统调用号:通过寄存器 a7 传递。

参数:前六个参数通过寄存器 a0 到 a5 传递。

返回值:通过寄存器 a0 返回(对于 64 位或更宽的返回

值,可能还使用 a1)。

四、 **XV6 (RISC-V) 代码级硬核细节 (必考)**

在 RISC-V 中，处理异常/中断的硬件动作非常精简，大部分工作依赖控制寄存器和软件（‘trampoline.S’）配合完成：

1. 核心控制寄存器 (Control Registers)

‘stvec’ (Supervisor Trap Vector Base Address Register)：保存陷入处理程序（如 ‘trampoline’ 或 ‘kernelvec’）的入口基地址。

‘sepc’ (Supervisor Exception Program Counter)：发生陷入时，硬件自动将被中断指令的 PC 值（即断点）保存在此，用于后续恢复。

‘scause’ (Supervisor Cause Register)：记录发生陷阱的原因（是中断、缺页、还是 ecall），OS 读取它来进行分发。

‘sstatus’ (Supervisor Status Register)：包含特权级状态、中断使能标志（如 SIE 位）。硬件会在陷入时自动更新此寄存器。

2. 上下文保存 (TRAPFRAME)

XV6 发生中断时，在 ‘trampoline.S’ 中，代码会将所有 32 个通用寄存器（包括 ‘sp’, ‘ra’, ‘a0~a7’ 等）全部保存到该进程专用的 ‘TRAPFRAME’ 结构体内存中。这与 x86 硬件自动压一部分入栈的行为有很大区别。

3. 源码部分

- 特权模式及启动转换 (Boot)
xv6-riscv 的特权模式：共有三种，分别是 机器模式 (M 模式)、监管模式 (S 模式) 和 用户模式 (U 模式)。
启动阶段转换过程：
M 模式 → S 模式：计算机加电后首先进入 M 模式。在 ‘start.c’ 的 ‘start()’ 函数中，硬件通过设置 ‘mstatus’ 寄存器并执行 ‘mret’ 指令，使 CPU 从 M 模式转换到 S 模式，随后进入 ‘main.c’ 执行。
S 模式 → U 模式：在 ‘main.c’ 完成系统初始化并创建第一个进程后，通过执行 ‘usertrapret’（最终在 ‘trampoline.S’ 中执行 ‘sret’ 指令），CPU 从 S 模式转换到 U 模式，开始运行用户程序。
- 系统调用寄存器传递 (Syscall)
在 xv6-riscv 中进行系统调用时，参数和信息的

传递遵循 RISC-V 调用约定：

系统调用号：通过 'a7' 寄存器传递。

参数：通过 'a0', 'a1', 'a2', 'a3', 'a4', 'a5' 寄存器传递（最多支持 6 个参数）

返回值：处理结果最后通过 'a0' 寄存器返回给用户程序。

3. 上下文切换保存的寄存器 (Trap)

根据 'trampoline.S' 和 'kernelvec.S' 的定义，当发生 Trap（如用户态进入内核态）时，需要保存进程的全套通用寄存器以确保现场可恢复：

具体寄存器包括：'ra' (返回地址)、'sp' (栈指针)、'gp' (全局指针)、'tp' (线程指针)、't0-t6' (临时寄存器)、'a0-a7' (参数/返回值寄存器)、's0-s11' (保存寄存器)。

保存位置：从用户态进入内核时，保存到该进程 PCB 的 'trapframe' 中；从内核态再次触发中断时，则压入内核栈。

4. 关键控制寄存器作用 (Trap)

阅读 xv6-book 4.1 节及 'trap.c' 代码，这些寄存器在 Trap 处理中起核心作用：

'stvec' (Supervisor Trap Vector Base Address Register)：保存中断处理程序的入口基地址。发生 Trap 时，硬件自动将 PC 指向此处。

'sepc' (Supervisor Exception Program Counter)：当发生 Trap 时，硬件自动将被打断的指令地址保存在此处，用于后续恢复执行。

'scause' (Supervisor Cause Register)：记录发生 Trap 的原因（例如是系统调用、缺页异常还是外部硬件中断）。

'sstatus' (Supervisor Status Register)：监控和控制当前处理器状态，包括中断使能标志 (SIE) 以及之前的特权模式 (SPP)。

5. 中断处理函数流程 (Interrupt)

'devintr' (设备中断分发)：首先读取 'scause' 寄存器确认是否为外部硬件中断。

调用 'plic_claim()' 获取当前发出中断请求的设备 ID。

根据设备 ID 跳转到相应的处理函数（如 'uartintr'）。

处理完成后调用 'plic_complete()' 告知 PLIC 处理完毕。

'uartintr' (串口中断处理)：读取 UART 的状态寄存器，判断是接收到了数据还是发送缓冲区已空。

如果有输入数据，则读取字符并将其送入控制台缓冲区 ('consoleintr')。

如果发送缓冲区空闲，则从输出环形缓冲区取下一个字符并发送。

'devintr' 调用 'uartintr' 的判断条件：

1. 发生的是外部硬件中断 ('scause' 显示为 Supervisor external interrupt)。

2. 通过 PLIC 识别出的设备 ID 与 UART0_IRQ (通常为 10) 匹配。

五、概念对比

中断发生后 OS 底层工作步骤

1. 硬件压栈：程序计数器等

2. 硬件从中断向量装入新的程序计数器

3. 汇编语言过程保存寄存器值

4. 汇编语言过程设置新的堆栈

5. C 语言中断服务程序运行（例：读并缓冲输入）

6. 进程调度程序决定下一个将运行的进程

7. C 语言过程返回至汇编代码

8. 汇编语言

第三章 操作系统核心模块：进程与线程模型

一、并发环境与进程概念

1. 顺序环境 vs 并发环境

顺序程序环境：具有封闭性（独占资源，不受外界影响）和可再现性（运行结果与执行速度无关）。系统中只有一个程序在运行。并发程序环境：一段时间内单处理器上有两个或以上程序开始运行且未结束。

核心特征（考点）：间断性（执行-停-执行）、资源共享、独立性与制约性、不可再现性（执行结果不再唯一确定）。

2. 进程 (Process) 的本质

经典定义：程序的一次执行过程，是对 CPU 的抽象。

核心功能：进程是具有独立功能的程序在某个数据集上的一次运行活动，是系统进行资源分配和调度的独立单位。

进程映像 (Process Image)：通常包含 程序代码 + 数据集合 + 栈 + 进程控制块 (PCB)。

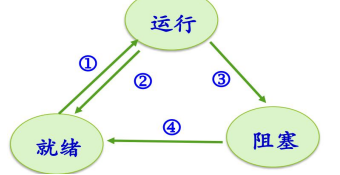
特性：并发性、动态性（动态产生消亡、在生命周期内三种基本状态转换、动态地址空间）、独立性（资源分配的独立单位，不同进程之间不影响）、制约性（可能与其他进程产生直接或间接关系、因访问共享数据/资源或进程间同步而产生制约）、异步性（以相对独立且不可预知的速度向前推进）

UNIX 进程家族树：init 为根；Windows：地位相同

二、进程状态流转与 PCB

1. 进程状态模型

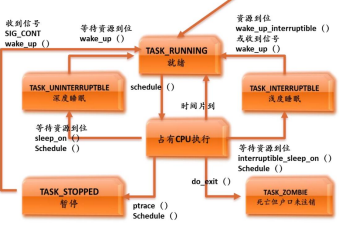
三状态模型 (最经典)：



五状态/七状态模型 (扩充)：

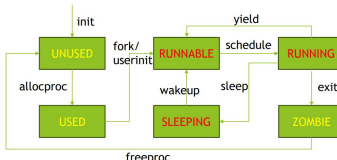
特殊 OS 的状态考点：

Linux 状态：'TASK_RUNNING' (就绪/运行态),



'TASK_INTERRUPTIBLE' (浅度睡眠), 'TASK_UNINTERRUPTIBLE' (深度睡眠), 'TASK_STOPPED' (暂停), 'TASK_ZOMBIE' (僵尸态：死亡但户未注销)。

xv6 状态：'UNUSED', 'USED', 'SLEEPING', 'RUNNABLE', 'RUNNING', 'ZOMBIE'。



2. 进程控制块 (PCB, 又称进程描述符、进程属性)

定义：操作系统中表示进程的数据结构，是系统感知进程存在的唯一标志。

PCB 存放在进程空间的内核数据段部分，相当于 cpu 对这个进程的句柄。

进程表：所有进程的 PCB 的集合。

Linux：task_struct；Windows：EPROCESS、KPROCESS、PEB（分了三块地方放 PCB）

四大核心信息块：

1) 进程描述信息：PID、进程名（通常基于可执行文件名）、User ID。

2) 进程控制信息：当前状态（enum procstate）、优先级、代码执行入口地址、可执行文件名（磁盘地址）、运行统计信息（执行时间/页面调度）、进程间同步和通信、阻塞原因、进程的队列指针、进程的消息队列指针。

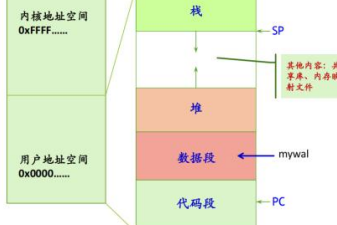
3) 所拥有的资源和使用情况：虚拟地址空间的现状、打开文件列表

4) CPU 现场信息 (Context)：通用寄存器值、PC、状态 PSW、栈指针、指向赋予该进程的段/页表的指针。

三、内存地址空间与进程控制

1. 进程的虚拟地址空间

构成（低地址到高地址，注意一般不是从 0x0 构成）



开始——防止野指针：代码段 (Text) -> 数据段 (Data) -> 堆 (Heap, 向上增长) -> 内存映射区/共享库 -> 用户栈 (Stack, 向下增长) -> 内核空间。

Linux 的内存数据结构：

顶层 'task_struct' 包含 'mm' 指针。'mm_struct' 包含硬件命脉 'pgd'（页全局目录基址，x86 体系中是 CR3 存放，给 MMU 用的）和软件命脉 'mmap'。

'mmap' 指向一系列 'vm_area_struct' 节点（链表），每个节点代表一块虚拟地皮（如数据段、代码段），记录了 'vm_start', 'vm_end' 和 'vm_prot' (读写权限)。

MMU 在翻译虚拟地址时，查到任一级页表项

的 valid=0 或权限不一样，则顺着 vm_area_struct 表寻找这段虚拟地址对应的 vm_area_struct，看看是否有关 COW 机制

malloc 的过程：

1. 判断分配大小：size≤阈值（128KB）用 brk 调整堆大小分配；size>阈值用 mmap 匿名映射分配

2. 小内存：

1) 找空闲链表，找到则直接切分使用不进内核

2) 找不到则调用 brk 扩展堆顶

3) 内核更新虚拟地址空间

4) 返回内存指针

3. 大内存：

1) 直接用 mmap

2) 在进程虚拟空间单独分配一块内存

3) 不在堆内（在 stack 下 heap 上的 shared libraries and huge malloc blocks）

4) 返回指针

进程表（队列）：队列为（就绪 1、就绪 2、..... 等待 1、等待 2.....），PCB 在不同队列中转换单 CPU 中运行队列只有一个 PCB 不同 PCB 等待不同的事件，进入不同的等待队列

2. 进程创建与撤销

原语 (Primitive)：具有不可分割性/原子性的操作，执行过程不允许被中断。

进程何时创建：系统初始化、操作系统提供的服务、交互用户登录系统、由现有的进程派生、提交一个程序执行（例如：命令行）

进程的创建：

给新进程分配一个唯一标识(pid)以及进程控制块(PCB)

为进程分配地址空间

初始化进程控制块

设置默认值（如：状态为 New,）

设置默认值的队列指针（如：把新进程加到就绪队列的链表中）

创建或扩充其他数据结构

UNIX 进程控制 API：

'fork()'：通过复制调用进程来建立新进程。

'exec()'：用一段新代码覆盖原来的内存空间，实现代码转换。

'wait()'：同步措施，等待另一个进程（通常是子进程）结束。

'exit()'：终止进程。

3. 核心考点：写时复制技术

问题：'fork()' 时如果把父进程内存一页页全拷贝，极度浪费空间和时间。

机制：

1) 'fork' 后，子进程获得 'mm_struct', 'vm_area_struct' 和页表的精确副本。

2) 内核将父进程的页表中的页面全部标记为只读 (Read-Only)。

3) 内核将父子进程的 'vm_area_struct' 标记为私有 COW。

4) 子进程状态设为就绪，插入就绪队列。

5) 缺页异常介入：当任何一方尝试写入时，硬件 MMU 拦截到写只读页的操作，触发异常。操作系统此时才会在物理内存中复制该页面，并将新页面的映射权限恢复为可写。

四、线程模型 (Thread Model)

1. 为什么引入线程？

分离进程的两大属性：进程原本既是“资源拥有者”又是“调度单位”。引入线程后，进程专注于资源的分配与保护（拥有虚拟地址空间），而线程成为 CPU 的调度单位和运行实体。

优势（考点）：

应用需求：如 Web 服务器可以利用多线程实现“接收请求”与“读取磁盘”并发执行，响应速度极快。

开销极小：线程创建、撤销远比进程快；同一进程的线程切换无需切换页表，极快。

通信便捷：线程之间共享内存和文件，通信无需调用内核。

性能：能充分利用多核、多处理器的并行计算能力。

2. 线程私有与共享的资源

私有资源：程序计数器 (PC) 及寄存器、用户栈、内核栈、线程控制块 (TCB)。

共享资源 (同进程内)：用户地址空间（代码段、数据段、堆）、打开的文件、全局变量。

3. 线程的实现机制 (三大模型)

1) 用户级线程 (ULT, User Level Thread)：

在用户空间通过线程库（如 POSIX Pthreads）实现，内核完全不知道线程的存在。

优点：切换无需陷入内核态，速度极快；调度算法可由用户自定义；可跑在任何 OS 上。

致命缺点：一个线程发起阻塞系统调用，内核会将整个进程挂起，导致同进程内所有线程全部阻塞；内核只分配一个 CPU 给该进程，多线程就无法利用多核 CPU 优势。

2) 核心级线程 (KLT, Kernel Level Thread)：

内核管理所有线程，维护线程上下文（如 Windows），线程切换需要内核支持。

特点：以线程为基础进行调度，支持多核并行，一个线程阻塞不影响其他线程。但切换需陷入内核态，开销比 ULT 大。

3) 混合模型 (Hybrid)：

线程创建在用户空间完成，调度在核心态完成。多个用户级线程多路复用多个内核级线程上（如 Solaris）。

4. 协程 (Coroutine) 与纤程 (Fiber)

协程：一种在用户态（不陷入内核）模拟的更轻量级的执行流，强调“协作式”而非“抢占式”放弃 CPU，用于改善高并发（Web、API 网关）、异步 I/O（数据库访问、MySQL 等客户端文件读写、爬虫请求）、UI 交互与事件驱动程序前端、批量并发请求聚合等的编程性能（如 Go 的 Goroutine）。

纤程：Windows 系统提供的机制，一个线程中可以有多个纤程。

第四章 进程与线程调度

零、核心概念：调度的三个层次

长程调度 (Long-term / 作业调度)：控制多道程序的“并发度”。决定新创建的进程是否进入当前活跃进程集合。

中程调度 (Medium-term / 交换调度)：涉及虚拟内存与物理内存的置换。将部分或全部进程换出(挂起状态)到外存以腾出内存空间，或将所需部分换入内存。

短程调度 (Short-term / 微观调度)：决定就绪队列中哪个进程/线程上 CPU 运行。时间尺度为“毫秒级”，要求极高的执行效率。

一、调度时机与上下文切换

触发时机：进程退出、异常终止、新创建进程、I/O 阻塞或等待资源、唤醒进程后重新回到就绪态、时间片用完回到就绪队列、执行 yield。Linux 和 xv6 中，当进程 read 阻塞，另一个进程执行 io 任务完成，将在另一个进程当前一条指令执行结束后处理中断，然后会重新调度进程。黄金法则：所有调度逻辑的真正执行，都发生在内核对中断/异常/系统调用处理完成后，返回到用户态前的最后时刻。

事件发生 -> 当前运行的进程暂停运行 -> 硬件机制响应后 -> 进入操作系统，处理相应的事件 -> 结束后处理：某些进程的状态会发生变化，也可能又出现了一些新的进程 -> 就绪队列有调整 -> 需要进程调度根据预设的调度算法从就绪队列选出一个进程

上下文切换 (Context Switch) 步骤：

1) 保存进程 A 硬件状态（PC、PSW、通用寄存器）至其 PCB（在 x86 中是软硬结合保存）。

2) 给进程 B 设置新状态，并将 A 的 PCB 移至合适的队列（就绪、阻塞）。

3) 把进程 B 的状态设置为运行态。

4) 切换全局项目录（CR3 -> x86/satp -> RISC-V）加载新进程 B 的物理地址空间。

5) 切换内核栈（修改栈指针寄存器存放的内容，进程 B 的栈指针在 PCB 中记录，注意这里是虚存）和硬件上下文，从新进程的 PCB 中恢复寄存器状态。

性能开销：直接开销为保存/恢复寄存器及切换地址空间的指令耗时；间接开销（致命）为 Cache、Buffer Cache 及 TLB（快表）的清空与失效重载。

调度算法的设计指标

1) 吞吐量：单位时间完成的进程数目。

2) 周转时间：从提出请求到运行完成的总时间。

3) 响应时间：从提出请求到系统给予第一次回应的时（交互式系统核心指标）。

4) 进程分类考量：I/O 密集型 (I/O-bound，需快速响应) 与 CPU 密集型 (CPU-bound，需大时间片)。

二、各类经典调度算法深度解析

0. 不同操作系统的目标

交互式进程（eg: shell、文本编辑程序）：响应时间、均衡性

批处理进程（eg: 利息计算等不必与用户交互且不必很快响应）：吞吐量、周转时间、CPU 利用率

实时进程（eg: 视频音频等有实时需求、不应被低优先级阻塞）：最后期限、可预测性

1. 先来先服务 (FCFS - First Come First Serve)

机制：按进入就绪队列的顺序调度，非抢占。考点：优点对长进程有利，缺点对短进程极不利。不利于用户的交互式体验。会导致 CPU 型进程长期霸占 CPU，使得后续 I/O 型进程无谓等待，降低设备利用率。

2. 短作业优先 (SJF) & 最短剩余时间优先 (SRTN)

机制：挑选预期运行时间最短的执行。SJF 为非抢占式；SRTN 为抢占式（新来更短的直接剥夺当前进程）。

考点：数学上能得出理论最优的最短平均周转时间。致命缺点是会导致长作业“饥饿 (Starvation)”，且在现实中极难准确预测进程未来需要的运行时间。

3. 最高响应比优先 (HRRN)

公式: \$响应比 R=1+ (作业等待时间 / 作业处理时间)\$

考点: 完美折衷。短作业由于处理时间极小, R 值大, 易被调度; 长作业随着等待时间线性增长, R 值也会逐渐变大, 最终被调度, 有效避免了饥饿。缺点: 需频繁计算, 资源开销大。

4. 时间片轮转 (RR - Round Robin)

示例: 4个进程的执行时间如下

P1	53
P2	8
P3	68
P4	24

时间片为20的RR算法示例



等待时间 P1=(68-20)+(112-88)=72
P2=(20-0)=20
P3=(28-0)+(88-48)+(125-108)=85
P4=(48-0)+(108-68)=88

平均等待时间 = (72+20+85+88)/4=66.25

机制: 排成循环队列, 每次执行一个时间片 (Quantum), 用完被时钟中断强制剥夺放到队尾。

考点: 时间片过长则退化为 FCFS; 过短则上下文切换开销占比过高。RR 对相同大小的进程其实“非常不友好”(会导致所有进程同步拖到最后时刻才完成, 平均周转时间甚至劣于 FCFS)。

5. 虚拟轮转法 (VRR - Virtual RR)

机制: 为每个进程设置一个固定的时间片, CPU 上时间片用完以后就切换。

优缺点: 有利于交互式计算、公平; 但轮转算法优势费较高开销, 不利好相同大小的进程。为了补偿由于 I/O 阻塞而提前放弃 CPU 的进程, 设立了一个辅助队列 (Auxiliary Queue)。考点: I/O 唤醒后的进程直接进入高优先级的辅助队列, 而不是普通就绪队列, 保证 I/O 密集型进程能及时获得 CPU 发起下一次 I/O。

6. 最高优先级调度

排序: 系统进程>用户进程; 前台进程>后台进程; 操作系统更偏好 I/O 型。

优先级反转问题:

现象: 高优先级 (H) 等待低优先级 (L) 释放锁, 而 L 又被中优先级 (M) 抢占, 导致 H 被实质性挂起。

解法: 优先级天花板协议 (把 L 临时提到最高)、优先级继承机制 (H 让出优先级给 L 使用)、关闭中断 (L 不允许被打断)。

7. 多级反馈队列调度 (MLFQ)

算法实现 (非抢占式):

设置多个就绪队列, 第一级队列优先级最高给不同就绪队列中的进程分配长度不同的时间片, 第一级队列时间片最小; 随着队列优先级别的降低, 时间片增大

当第一级队列为空时, 在第二级队列调度, 以此类推

各个队列按照时间片轮转方式 进行调度 当一个新创建进程就绪后, 进入第一级队列 进程用完时间片而放弃 CPU, 进入下一级就绪队列 (被时钟中断强行叫停 trap)

由于阻塞而放弃 CPU 的进程 (主动 yield) 进入相应的等待队列, 一旦等待的事件发生, 该进程回到原来一级就绪队列

当有一个优先级更高的进程就绪时, 可以抢占,

被抢占的进程回到原来一级就绪队列末尾

3、真实操作系统调度实现拆解

1. Windows 线程调度机制

基本架构: 基于 32 级动态优先级 的抢占式多任务调度, 结合时间配额 (Quantum) 调整。

16~31 级: 实时优先级 (绝对不改变, 一般是操作系统线程)。

1~15 级: 可变优先级 (根据行为动态升降)。

0 级: 系统零页线程 (仅在无其他线程时运行, 负责物理页清零)。

数据结构: 维护 32 位的就绪位图 (Ready Bitmap), 以及一个 32 位的空闲位图, 每一位指示一个处理机是否处于空闲, \$O(1)\$时间内找到最高非空闲队列。

调度策略: (优先级改变/亲和和处理机改变时触发调度)

进程在运行态主动 yield 进入阻塞态。

抢占: 高一级的进程从阻塞态被唤醒 (中断处理后修改状态), 原有运行的低级进程回到原级别就绪队列的队首。处于实时优先级的, 时间配额重置为完整; 可变优先级继续运行剩余的配额。

时间配额用完: A 优先级没降低——原队列有其他进程则将 A 放置队尾, 否则 A 继续运行; A 优先级降低了——将选择一个优先级更高的进程。

优先级动态提升 (Bonus):

1) I/O 操作完成时 (临时提升等待该操作线程的优先级, 并在唤醒等待线程时把时间配额减一, 响应要求越高, 提升幅度越大)。

2) 等待事件/信号量结束时 (提升该线程一个优先级但不会超过 15 级, 时间配额减一, 在提升后的优先级上运行完剩余时间配额, 降低一个优先级, 运行新的时间配额, 直至降低到基本优先级)。

3) 前台进程中的线程完成了一个等待操作。

4) 前台窗口活动被唤醒时 (eg: 鼠标点击)。

防饥饿: 平衡集管理器 (Balance Set Manager) 每秒扫描, 排队超 300 个时钟间隔的线程, 被强行提升至 15 级并赋予 4 倍时间配额, 用完后立即衰减回原点。

空闲线程: 如果有待处理的 DPC 请求, 清除相应软中断并执行 DPC; 若线程就绪则调度相应线程: 执行相应的电源管理功能。

2. Linux 调度器的史诗级演进

这是考试极容易考察的内核演进脉络!

Linux 2.4 调度器 (O(N) 遍历)

单队列, 基于 counter+优先级, 越低越优先。当所有就绪进程的 counter (时间片) 都耗尽后, 产生一个 \$O(N)\$ 的全员重新计算过程。

睡眠进程醒来时, 保留其未用完的 counter 并叠加新配额, 借此提升交互进程优先级。性能和扩展性极差。

实时进程的 counter 用完之后会立刻重置 counter 然后放入就绪队列, 而非实时进程要等待 runqueue 中为空时, 统一重新计算 counter

Linux 2.6 O(1) 调度器

设计双数组: active (存很多优先级队列) 和 expired (过期队列)。

选取 active 中最高优先级, 进程时间片耗尽, 若为交互式进程或实时进程, 则重置时间片并重新插入 active 中, 否则直接扔进 expired。当进程已经占用 CPU 足够长时, 就全部扔进 expire。

当 active 为空时, 仅需交换两个数组的指针, 实现 \$O(1)\$ 开销。

依靠极其复杂的经验公式 (睡眠时间越长, bonus 越大) 来给交互式进程计算 Bonus, 实时进程的优先级不会动态修改, 总比普通进程高。

SD (Staircase Scheduler) 楼梯调度

对于实时进程还是 FIFO/RR 策略。

时间片用完降级到 active 数组中低一优先级的队列中。降到底部后回到原始级别的下一级, 并获得 N+1 倍时间片。

RSDL (Rotating Staircase Deadline)

给每一个优先级设立了“组时间配额 (Tg)”。一旦高优先级组的 Tg 耗尽, 无论组内个别进程的时间片 (Tp) 是否用完, 全组被强制踢下楼梯 (Minor Rotation)。此外, 一个进程自己的时间片用完, 放到同级别的 expired 数组中, 若所有进程都用完饥饿者都到了最低层, 则交换。使得底层的饥饿进程能在一个“可精确预期的 Deadline 内”获得调度权。

CFS (Completely Fair Scheduler) 现代 Linux 王者

核心思想: 彻底废除静态时间片, 按权重比例分配 CPU 物理时间。核心公式: 虚拟运行时间 (vruntime) = (调度周期 * 进程权重 / 所有进程总权重) * NICE_0_LOAD / 进程权重。

数据结构: 所有进程被挂载在红黑树中, 调度器永远挑选最左下角 (vruntime 最小) 的节点执行。

唤醒补偿: 维护一个全局变量 min_vruntime, 睡眠进程唤醒时, 将其 vruntime 补偿至接近 min_vruntime 的值, 使其一醒来就因值较小而立刻抢占 CPU。

3. 多核与实时调度机制

实时调度: 可调度性前提: \$sum_{i=1}^n (C_i / P_i) \le 1\$ (\$C\$ 为处理耗时, \$P\$ 为事件 \$S\$ 发生的周期)。

RM (速率单调): 静态优先级, 执行时间越短优先级越高。

EDF (最早截止时间优先): 动态优先级, 距离 Deadline 越近优先级越高。

多核机制:

1) 缓存亲和性 (Cache Affinity): 调度器尽量将进程绑定在曾经运行过它的物理 Core 上, 避免 L1/L2 Cache 大面积失效 (如 Go 语言 GMP 调度器机制)。

2) 负载均衡机制: openEuler 系统使用专门的迁移线程 (migration/CPUID) 强制将进程从过载 CPU 的队列“窃取 (Work Stealing)”至空闲 CPU 队列中。

3) 缓存一致性: 有两个处理器同时读取并修改共享数据 (解决: MESI 协议)

YARN 调度流程: 1) 客户端提交任务→RM 2) RM 给这个应用启动一个 AM 3) AM 向 RM 申请资源 (container) 4) RM 把可用节点/资源返回给 AM 5) AM 命令对应节点的 NM 启动 container 运行任务 6) NM 监控执行, AM 监控整个应用 7) 结束→AM 退出, 资源回收

3.2.2 内存管理

一、内存管理的核心认知与目标补充 核心概念澄清: 内存管理绝对不等于虚拟内存管理, 虚拟内存只是内存管理发展到后期的一项高级技术。

管理机制的演化过程 (三个主要阶段):

- 1) 早期物理内存使用。OS 直接管理唯一的连续物理内存, 属于单道程序设计。
- 2) 多道程序设计与分时共享。允许多个程序同时进入内存, 引入了独立的进程地址空间, 此时必须解决“地址重定位”和“存储保护”两大核心问题。
- 3) 虚拟内存。引入了局部性原理和分页置换机制。
- 2、进程地址空间与绑定、重定位机制 进程地址空间的具体内容与图示细节



XV6 操作系统的地址空间特殊布局考点

1) 基础布局: 地址从低到高依次为 text, data & bss, fixed-size stack (固定大小栈), expandable heap (可扩展堆)。

2) 特殊映射机制: 高地址处存放有 trampoline 和 trapframe, 这两个特殊区域在用户态和内核态都做了映射。

3) 宏定义细节考点: TRAMPOLINE 映射到虚拟地址空间的最高端 (MAXVA 减去 PGSIZE 的大小); 内核栈对应的宏 KSTACK(p) 映射在 trampoline 之下, 为了安全, 每个进程的内核栈周围都设置了无效的保护区 (invalid guard pages) 以防止栈溢出越界; TRAPFRAME 则映射在 TRAMPOLINE 下方紧邻的一个页面 (TRAMPOLINE 减去 PGSIZE)。

指令与数据的绑定时机 (Binding Time)

1) 编译时 (Compile time): 如果已知程序将驻留在内存的哪个绝对位置, 编译器可直接生成绝对物理代码。

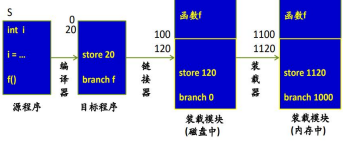
2) 链接时 (Linking time): 在多模块链接阶段确定相对地址。

3) 装载时 (Load time): 如果编译时无法确定物理地址, 编译器必须生成可重定位代码, 由装载器在加载进内存时完成物理地址的绑定。

4) 运行时 (Run time): 如果进程在执行期间可以在内存不同位置之间移动 (如发生交换), 则绑定必须延迟到指令实际运行时执行, 这需要底层硬件 (MMU) 的绝对支持。

地址重定位的具体过程对比

静态地址重定位: 由装载器 (软件层) 在用户程序加载到内存时, 一次性实现逻辑地址到物理地址的转换。例如代码中的“store 120”, 如



果装载模块的基址是 1000, 装载器会直接将指令改写为“store 1120”。

动态地址重定位: 在进程的整个执行过程中,



即逐条指令执行时瞬间完成地址映射。依赖 CPU 内的内存管理单元 (MMU)。CPU 发出逻辑地址 (如 456), MMU 将其加上重定位寄存器中保存的基地址 (如 12000), 瞬间输出物理地址 (12456) 以访问内存。

地址保护的具体硬件流程

机制支撑: 主要依靠基地址寄存器和界限寄存器配合完成。

执行逻辑: 操作系统通过特权指令在进程切换时将当前进程的范围加载进这两个寄存器。CPU 执行指令时, 首先检查发出的地址是否“大于等于基地址”; 如果是, 再检查该地址是否“小于 (基地址加上界限地址之和)”。

如果这两个条件有任何一个不满足, 硬件立刻判定为地址越界, 并陷入 (Trap) 操作系统终止非法访问。

三、空闲区数据结构

位图法: 将内存按固定分配单元划分, 位图中的一位对应一个单元, 0 表示空闲, 1 表示占用。

空闲区表/已分配区表: 适用于不等长划分。表中详尽记录每块内存的“起始地址、长度、标志”。

空闲块链表: 包含隐式空闲链表、显式空闲链表以及分离空闲链表。

衡量标准: 外碎片 (块与块间)、内碎片 (页二、连续内存分配算法与回收处理)

四大分配算法:

1) 首次适配 (First fit): 从头遍历空闲区表, 找到第一个满足进程大小要求的空闲区即分配。

3) 下次适配 (Next fit): 从上一次找到的空闲区位置开始接着向下查找。

4) 最佳适配 (Best fit): 遍历整个空闲区表, 找到能够满足要求且“最小”的空闲区进行分配, 以保留大块内存。

4) 最差适配 (Worst fit): 总是分配满足要求的“最大”空闲区, 剩余部分切分为新的空闲区。

“分离适配”算法

策略名称	特点
Simple Segregated Storage	不合并、不分割、链表。嵌入式实时系统 (确定性延迟)
Segregated Fit	允许分割大块, 但释放时可能合并。dmalloc (Doug Lea malloc)
Buddy System (伙伴系统)	仅允许 2 的幂次大小, 合 Linux 内核内存分配, GPU 内存管理
Slab Allocator	针对数据类型, 而非 Linux 内核 malloc, 对象缓存
Page Attention (页级注意)	未分配成 Block, 而是 VLLM KV Cache 管理

内存回收时的相邻合并规则 (四种情况考点): 当一块内存归还时, 系统必须检查并修改空闲区表。若回收区与前面的空闲区相邻 (上相邻)

或与后面的空闲区相邻 (下相邻), 甚至上下都相邻, 必须将它们合并成一个大空闲区; 若上下都不相邻, 则作为新条目插入表内。

三、伙伴系统 (Buddy System) 详细细演 核心设计思想: Linux 底层物理内存管理的基石。将内存按 2 的幂 (如 2 的 U 次方) 进行划分, 组成多个不同大小的空闲块链表。属于一种特殊的“分离适配”算法。

拆分与分配规则: 假设进程申请空间为 s, 如果 2 的 (U-1) 次方 小于 s 且 s 小于等于 2 的 U 次方, 则直接分配整个 2 的 U 次方大小的块; 否则, 将当前块等分为两个大小为 2 的 (U-1) 次方的“伙伴”块, 一直递归划分直到产生刚刚好大于或等于 s 的最小块。

动态推演考点示例 (假设总内存为 1MB): 1) 进程 A 申请 100K: 系统将 1MB 分为两个 512K, 再取一个 512K 分为两个 256K, 再取一个 256K 分为两个 128K。将其其中一个 128K 分配给 A。

2) 进程 B 申请 240K: 直接从上一步剩下的 256K 空闲块中分配。

3) 进程 C 申请 64K: 系统取剩下的那个 512K 块, 切分为 256K, 再切出 128K, 再切出两个 64K, 将其中一个 64K 分配给 C。

4) 合并机制: 当进程释放内存时, 如果其互为“伙伴”的相同大小内存块也是空闲的, 系统会自动将它们合并为高一级的较大内存块。

四、Linux 小内存分配器家族 (SLAB 演进史) SLAB 分配器: 功能: 专门解决操作系统频繁分配“小对象” (通常是 2 的三次方到 2 的十二次方字节之间) 的需求。把大块内存进一步细分, 并使用 slab cache 作为本地高速缓存, 对象释放时优先放入缓存中。

缺陷: 由于维护了太多队列, 代码日趋复杂, 在高并发多核环境下极易出现锁竞争导致性能下降。

SLUB 分配器: 2007 年由 Pekka Enberg 开发, 被合并入 Linux 2.6.22 版本。采用简化的数据结构大幅减少了锁竞争, 并专门针对多核及多节点 NUMA 架构做出了本地性优化, 提升了可伸缩性。

SLOB 分配器: 最早出现在 Linux 2.2 版本中。其设计目标是极致的简单和紧凑, 主要为了减少代码尺寸和内存开销, 专为小型嵌入式系统和资源严重受限的环境设计。

第三部分: 各种基本内存管理方案详细对比 一、连续分配方案及其缺陷

单一连续区: 每次只运行一个用户程序, 用户程序独占除系统外的所有内存。

固定分区: 把内存空间分割成若干个连续区域 (大小可同可不同), 但分区大小固定不变, 每个分区只能装载一个进程。支持单队列或多队列。

可变分区: 根据进程实际需求动态切分内存, 会引发严重的“外碎片”问题 (即产生大量极小且不易利用的空闲区, 导致内存利用率下降)。

紧缩技术 (Memory compaction): 为了解决可变碎片, OS 在内存中移动程序, 将所有小碎片强行合并为一块大空闲区。此方案系统开销巨大, 需要严谨评估移动时机 (I/O 时不行)。

二、离散分配方案深入拆解 页式内存管理 (Paging)

(1) 核心概念: 逻辑上将用户程序按相等大小

划分为“页——4KB（Page）”，物理内存按同样大小划分为“内存块/物理页面/页框（Page Frame）”。此划分由系统自动完成，对用户透明。

- （2）碎片情况：消除了外碎片，但不可避免地产生“内碎片”（程序最后一页通常无法填满）。
- （3）**地址结构与转换考点**：典型的逻辑地址被强行划分为两部分。高位为“页号”，低位为“页内偏移”。（例如 32 位地址中，0-11 位为偏移即 4096 个地址，12-31 位为页号，可到达 0 到 1048575）。CPU 用到逻辑地址后，利用页号去查询位于内存中的“页表”，获取对应的物理页框号，再与页内偏移拼接为最终物理地址。

段式内存管理（Segmentation）

- （1）核心理念：按照程序自身的逻辑结构（如主程序段、子程序段、栈段、数组等）划分为若干个名字不同的段，各段长度各异。物理内存随之被动态划分为不等长的物理段。对用户可见。
- （2）特性：以段为分配单位，每个段内部物理空间必须连续，但段与段之间可以分散存放。
- （3）**地址结构与转换考点**：逻辑地址包含“段号”和“段内地址”。CPU 通过段号查询“段表”，段表中不仅记录了物理段的“起始地址”，还记录了该段的“段长度”。CPU 不仅计算基址加偏移，还会用段长度进行越界检查。
- 段页式内存管理（Segment-Paged）**
- （1）背景：结合了页式的消除外碎片与段式的逻辑清晰、易于保护共享的优点。（2）结构重塑：用户程序依然按照逻辑段划分，但底层的物理内存按照页式划分。分配单位变成了“页”。（3）数据结构嵌套考点：系统为每个程序维护一个“段表”，但此时段表里记录的不再是物理基址，而是该段“页表”的起始“和“页表的长度”。即每个进程有一个段表，同时可能有多个页表。（4）逻辑地址：分为三段——段号、页号、页内地址/偏移。
- 第四部分：内存扩充技术深层探讨

一、覆盖技术（Overlaying）

解决痛点：在可用物理空间较小时运行超出内存总和的大型程序。

运行机制：依据程序的逻辑层次结构，让那些肯定不会同时执行的模块共享同一块物理内存区域，在需要时互相覆盖覆盖。

调用结构图示：例如某程序总计需要 54K。其中主控模块 A 常驻（8K）；模块 B（8K）与模块 C（10K）互斥；模块 D（12K）、E（4K）、F（10K）位于更底层且互斥。

系统只需开辟：常驻区 8K + 覆盖区一（取 B 和 C 最大值 10K）+ 覆盖区二（取 D,E,F 最大值 12K），即可将占用的模块内存压缩到 30K。

核心缺陷：对用户毫不透明，必须由程序员自己梳理划分功能模块并写明覆盖关系，大幅增加了编程难度，且从外存频繁调入模块会显著增加执行时间（用时间换空间）。

二、交换技术（Swapping）细节考点

核心理念：最早用于小型分时系统（Roll in / Roll out）。当内存紧张时，OS 将某些进程暂时移到**磁盘外存**（换出），并把外存中急需运行的进程调入内存（换入）。

交换区（Swap Space）：一般操作系统会在磁盘上单独指定一块包含连续磁道的特殊区域作

为交换空间，它允许 OS 绕过常规文件系统，使用底层磁盘读写接口对其进行最高效的块访问。

交换策略与时机考点：

1）**交换对象**：运行时动态创建或修改的内容（如栈段和数据段/堆）必须被完整保存到交换区。只读的代码段在许多系统中直接丢弃，用时重新从可执行文件载入即可。

2）**换出原则**：通常与调度器结合。处于等待 I/O 状态的进程往往优先被换出，以免抢占内存资源。

3）**进程空间增长的处理困难**：进程在换入换出期间，其数据段和栈段可能由于运行而变大。为解决此问题，操作系统在分配物理内存或交换区时，通常会预留在数据段和栈段之间预留一定的“**增长空间（Room for growth）**”。如果增长超过了预留，就必须给该进程寻找更大的内存空洞重新安置。后面就改成 stack 向下 heap 向上。

4）**重定位关联**：进程被换出后再换回内存时，**大概率不会回到原来**的物理位置，这必须依赖上文提到的动态地址重定位（MMU）技术来保障程序的正常运行。

- 寻址逻辑**：
- SATP（RISC-V）在不同的进程之间是不一样的，它存储了真正的物理地址，以便 MMU 在不同的进程中进行虚拟->物理的转换过程。
- 而在每个进程中，所有给进程/cpu 看的地址都是虚拟地址，只有在真正用到，比如说 pc 指向这条指令，然后取指过程中，把 pc 的虚拟地址转换为物理地址，硬件点对应的物理地址处把指令取回来。
- 栈指针寄存器在 cpu 中只有一个，但一个进程有两个栈，用户和内核栈，在权限切换时，这个栈指针会改变指向（这是软件做的）。
- pc 也只有一个，在切换权限时，硬件将用户态的 pc 保存在 trapframe 中，随后在 trampoline.s 里，软件把其他的寄存器也保存在 trapframe 中。
- 接下来进行内核态的代码（这部分代码存储在内核态的那部分虚存中，这部分内容是所有进程的内存态共享的）。
- 执行到需要调度时，内核执行一个 swtch，将一部分寄存器的内容存到这个进程的 pcb 下的 context 里，而由于调用了 call，会将准备返回（也就是调度回来以后下一条该执行的指令）的指令存入 ra，在汇编代码中将 ra 也一并存入 context，然后交出栈指针和 pc，给另一个进程的内核使用，另一个进程的内核态从它的 pcb 里读出 context 并填入相应的寄存器，恢复上下文并开始继续运行。

三、COW：写时复制

这里的 Private 小块指的是 P1 和 P2 共享的若干虚拟页（P2 会得到一个完全不同的一级页表基址，这个一级），它们映射在 Physical memory 的真实物理页上。fork 时，这些页的权限，在页表上被标记为只读（这样下次写的时候会触发缺页），而 fork 时，P2 会得到与 P1 完全一样的对应这些页的 VMA 结构体（比方说这个 VMA 表示的是.text 段），并且把这些 vma 中的 flags 改为 private COW。P2 下次写其中一页时

（黄色块代表最少一页），MMU 找到这一物理页的权限是只读，触发缺页异常，内核从 vma 链表里找到这个虚拟地址对应的(vm_start, vm_end)段，一看其实是写时复制，于是分配一个新的物理页给 P2 这次写操作，并且把这个新的物理页设置为可读可写。

虽然页表基址不一样，而且 P1 和 P2 也会有完全不重叠的物理空间上的两组多级页表，但是由于共享虚拟地址，它们的寻址逻辑实际上是一样的，只是一开始找的第一级页表的物理位置不一样，导致他们后续找到的页表物理位置也不一样，但虚拟地址决定着每个页表中要找的索引是一样的。所以实际上，P1 和 P2 的前几级每个页表物理地址不一样，存的条目（指向下一级页表地址）也不一样，只有最后一级页表，他们存的条目是一样的（共享物理页）。

RISC-V 指令集双进程调度与寻址：

- 零、世界观设定（初始状态）
- 在这个系统中，只有一条真实的 8GB 物理内存条（RAM）。但有两个进程，它们各自拥有独立的 4GB 虚拟地址空间。
- 物理内存 (RAM)：真实宇宙。
- Process A (PID 5)：正在执行 read(fd, 0x4000, 4096)，试图把文件读到虚拟地址 0x4000。
- 2.1 它的页表基址寄存器（SATP）指向物理页：0x80001000。
- 2.2 它的内核栈（kstack）物理地址：0x1000A000。Process B (PID 6)：即将执行 mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0)，目标是映射到虚拟地址 0x6000。
- 3.1 它的页表基址寄存器（SATP）指向物理页：0x80002000。
- 3.2 它的内核栈（kstack）物理地址：0x1000B000。
- 一、第一幕：Process A 的真实寻址（系统调用执行中）
- 陷入内核：
- A 执行了 read 的 ecall 指令。硬件将特权级切为内核态，A 的 32 个用户寄存器被汇编代码压入 A 的 trapframe 中。CPU 栈指针（SP）切换到 A 的内核栈 0x1000A000。
- MMU 的翻译（内核态写数据）：
- 内核拿到了用户传进来的目标虚拟地址 0x4000。内核调动磁盘控制器，准备把 4KB 的文件数据写进去。此时，MMU（内存管理单元）介入，开始查 Process A 的页表（根据 SATP = 0x80001000）：
- 2.1 MMU 提取虚拟地址 0x4000 的虚拟页号（VPN）：0x04。
- 2.2 MMU 在 A 的页表中查找第 0x04 项，发现对应的物理页帧号（PFN）是 0xF200，且权限合法（PTE_V | PTE_W | PTE_U = 1）。
- 2.3 MMU 将 PFN 和偏移量结合，生成真实物理地址：0x02F00000。
- 2.4 磁盘 DMA 将文件数据疯狂灌入物理内存条的 0x02F00000 处。进程 A 毫不知情，它以为数据进到了 0x4000。
- 二、第二幕：时钟中断与夺舍（上下文切换）
- 时钟滴答（Timer Interrupt）：
- 突然！主板上的时钟芯片发出硬中断。正在 A 的内核栈里干活的 CPU 被强行打断。内核意识到：A 的时间片用完了。
- 封存 A 的进度（保存 Context）：

内核调用 yield()-> sched()-> swtch()。swtch 汇编函数把 CPU 当前的内核寄存器（比如 ra 返回地址、当前的内核栈指针 sp = 0x1000A800）保存到 A 的 PCB 档案袋中，即 proc[5].context。

唤醒 B 的进度（恢复 Context）：

调度器（Scheduler）在 PCB 数组中选中了 RUNNABLE 状态的 Process B (PID=6)。swtch 函数从 proc[6].context 中抽出 B 上次休眠时保留的内核寄存器，强行覆盖到 CPU 硬件寄存器上。最核心的一步：内核执行一条特殊指令（向 SATP 寄存器写入 0x80002000），瞬间切换了 MMU 的翻译字典！从这一刻起，CPU 眼中的整个虚拟空间变成了 B 的宇宙。

三、Process B 的（mmap 全过程）

B 回用户态并调用 mmap：

B 被调度执行后，一路从内核栈返回到了用户态。B 执行了你的代码：mmap(addr=0, length=4096...)。陷入内核。

sys_mmap 的“虚空间饼”（Lazy Allocation）：

内核进入 B 的 sys_mmap 函数。内核在 B 的虚拟空间高处找到了一个空地（比如虚拟地址 0x6000），并在 B 的 VMA 账本（proc[6].vmas）中登记：

7.1 v->addr = 0x6000, v->length = 4096。

7.2 注意！此时没有任何物理内存被分配！B 的页表里虚拟页 0x06 依然是无效的（PTE_V = 0）。内核直接把 0x6000 这个字符串当做返回值去给用户态的 B。

B 触雷（Page Fault 缺页中断）：

B 回到用户态，兴奋地执行了一句 p[0] = 'X'（试图向虚拟地址 0x6000 写数据）。MMU 拦截：查 B 的页表（SATP=0x80002000），发现第 0x06 项是空的！MMU 立刻向 CPU 抛出异常（Page Fault）。B 再次被迫陷入内核态。

内核填坑（分配物理页）：

内核的 usertrap 捕捉到了缺页异常，读取 stval 寄存器，发现惹祸的虚拟地址是 0x6000。内核去查 B 的 VMA 账本，发现 0x6000 确实是刚才 mmap 登记过的合法范围！于是，内核：

- 9.1 从空闲物理内存池中找出一页，物理地址为 0x03F00000。
- 9.2 调用文件系统接口，把底层文件的 4KB 数据读进物理页 0x03F00000。
- 9.3 修改 B 的页表：把虚拟页 0x06 映射到物理页框 0x3F00，设置标志位 PTE_V=1。
- 时空倒流（重试指令）：
- 坑填好了，内核执行 sret 返回用户态。硬件自动重新执行刚才失败的那句 p[0] = 'X'。这一次，MMU 查表顺利通过，将 0x6000 翻译为真实的物理地址 0x03F00000，字母 'X' 被完美地写进了真实的内存条中。



附录：跨体系调度策略对比一览表

调度算法名称(中英文)	英文缩写	调度决策方式	核心理念/优先级调度方式	适用场景 / 优缺点特征
先来先服务 First Come First Served	FCFS	非抢占式	绝对按进入就绪队列的时间先后排队。	对长作业极度有利，极度压榨 I/O 密集型程序，不利于交互。
最短作业优先 Shortest Job First	SJF	非抢占式	预估运行时间最短的优先调度执行，直到阻塞或完成。	拥有理论最低平均周转时间，但容易导致长作业饿死。
最短剩余时间优先 Shortest Remaining Time Next	SRTN	抢占式	SJF 的抢占版本，新到达任务如果比当前任务剩余时间短，立即调停。	为短进程提供极佳响应度，但强依赖运行时间预测。
最高响应比优先 Highest Response Ratio Next	HRRN	非抢占式	动态计算响应比 $R = 1 + (WaitTime / ServiceTime)$ 。	完美折衷：短任务天生 R 高；长任务等待时间随增长 R 变高，杜绝饥饿。
时间片轮转 Round-Robin	RR	抢占式(带时间片耗尽)	系统划分时间片配额 q，所有进程同等分配，耗尽即阻塞入队。	对交互计算公平友好；缺点是时间片过小，耗尽时间片向下降级。
虚拟轮转 Virtual Round-Robin	VRR	抢占式	RR 的变种，为 I/O 阻塞就绪的进程设立专有优先级辅助队列。	解决 RR 无法充分照顾 I/O 密集型进程使其丧失 CPU 利用率的痛点。
最高优先级优先 Highest Priority First	HPF	抢非抢占皆可	赋予静态或动态优先级，严格优先级调度最高级。	易导致低优先级关键任务及“优先级反转”现象。
多级反馈队列 Multilevel Feedback Queue	MLFQ	抢占式	设立多级队列，q1 优先级最高时间片较短，耗尽时间片则向下降级。	现代 OS 的根基。完美平衡交互与长计算：短任务在顶层迅速完成，长任务在底层欠次时间片。
完全公平调度器 Completely Fair Scheduler	CFS	抢占式	彻底抛弃队列，构建红黑树。按权重分配 CPU 虚拟运行时间（vruntime）。	Linux 现任王者。用绝对精确的数学比例实现真正的时间公平。
速率单调调度 Rate-Monotonic	RM	抢占式(静态)	实时系统专属。任务周期 (Period) 越短，赋予的静态优先级越高。	经典硬实时调度，易于实现验证，基于强周期性假设。
最早截止时间优先 Earliest Deadline First	EDF	抢占式(动态)	实时系统专属。距离 Deadline 最近的任務周期获得最高优先级。	动态计算开销大，但在可行性条件下能保证 100% 任务按期完成。
工作窃取调度 Work Stealing	-	多核均衡机制	空闲 CPU 主动去繁忙 CPU 的待执行队列末尾“偷”任务执行。	多核时代的并行王者算法，平衡了 Cache 亲和性与 CPU 整体吞吐量。

概念维度	Intel x86 (IA-32 / x86-64)	RISC-V (以 xv6 实验环境为例)	核心机制与区别解析
特权级别 (Privilege Levels)	4 Ring 架构 (Ring 0 ~ Ring 3)。OS 常带 Ring 0 (内核) 和 Ring 3 (用户)。	3 Mode 架构 (M, S, U)。M 是机器态(最高权，常驻组件)，OS 内核跑在 S 态，用户跑在 U 态。	RISC-V 引入 M 态是为了支持底层的硬件器件(SBI)和虚拟化基础，而 OS 通常只拥有 S 态权限。
页表基址寄存器	CR3 (Control Register 3)	SATP (Supervisor Address Translation and Protection)	切换进程虚存的关键 。发生进程调度时，OS 必须将新进程的页表根目录物理地址写入该寄存器。SATP 还能配置页表模式 (Sv39 等)。
中断向量表/描述符表	IDT (Interrupt Descriptor Table)，通过 IDTR 寄存器定位。	统一统入口 stvec，不查硬件表。	设计哲学差异 ：x86(CISC) 硬件极其复杂，硬件根据中断号查 IDT 表直接跳转到对应的服务程序。RISC-V(RISC) 硬件极简，所有陷阱统一跳转到 stvec 指向的同一地址，由软件去读 scause 进行查表分类。
中断返回地址保存	硬件自动把断点 (EIP/CS) 压入内核栈 (Memory)。	硬件自动把断点存入 sepc 内部寄存器。	RISC-V 为了提高速度，陷入内核的一瞬间绝不访问内存，全靠内部寄存器留证。
系统调用指令	早期：int 0x80 现代：sysenter / syscall	ecall (Environment Call)	ecall 会触发同步异常，硬件将 scause 设为对应的值（如来自 U mode的scal为0）。
系统调用传参方式	eax 传调用号。ebx, ecx, edx, esi, edi 传参数。	a7 传调用号。a0 ~ a5 传参数。	原理相同，都是因为陷入指令无法携带复杂参数，所以借用通用寄存器作为载板。
陷门 vs 中断门	EDI 中明确区分 Interrupt Gate (自动关断) 和 Trap Gate (不关中断)。	硬件不分门类。陷入内核瞬间，硬件自动清除 status 中的 sie 位（统一自动关中断）。	RISC-V 交给内核软件自己决定何时重新开启中断，硬件行为更加统一。
上下文保护 (Context Save)	软硬混杂：硬件自动压入 SS, ESP, EFLAGS, CS, EIP。剩下的由汇编 SAVE_ALL 压栈。	纯软件干活：汇编代码在 trampoline.s 中手动将 32 个寄存器保存到内存的 Trapframe 结构中。	x86 有硬件上下文压栈支持；RISC-V 坚持 RISC 理念，让汇编代码通过普通的 sd 指令完成所有的现场保存。

概念描述	RISC-V 体系结构	x86 / x86-64 体系结构	核心作用说明
页表基址寄存器	satp (Supervisor Address Translation & Protection)	CR3 (Control Register 3)	存放当前进程的 顶级页表物理地址 。每次进程切换时必须写入此寄存器，MMU 凭它进行地址翻译。
中断向量入口	stvec (Supervisor Trap Vector)	IDTR (Interrupt Descriptor Table Register)	告诉 CPU：发生中断/异常时，操作系统的“应如何处理大门”（入口函数地址）开在哪里。
缺页异常地址存放	stval (Supervisor Trap Value)	CR2 (Control Register 2)	当发生 Page Fault（缺页异常）时，硬件会自动把“惹祸的那个虚拟地址”存放在这里，供 OS 读取填坑。
保存断点地址	sepc (Supervisor Exception PC)	硬件自动压入内核栈 (EIP/RIP)	发生中断的一瞬间，硬件保存当前被打断的指令地址，以便将来返回时继续执行。